

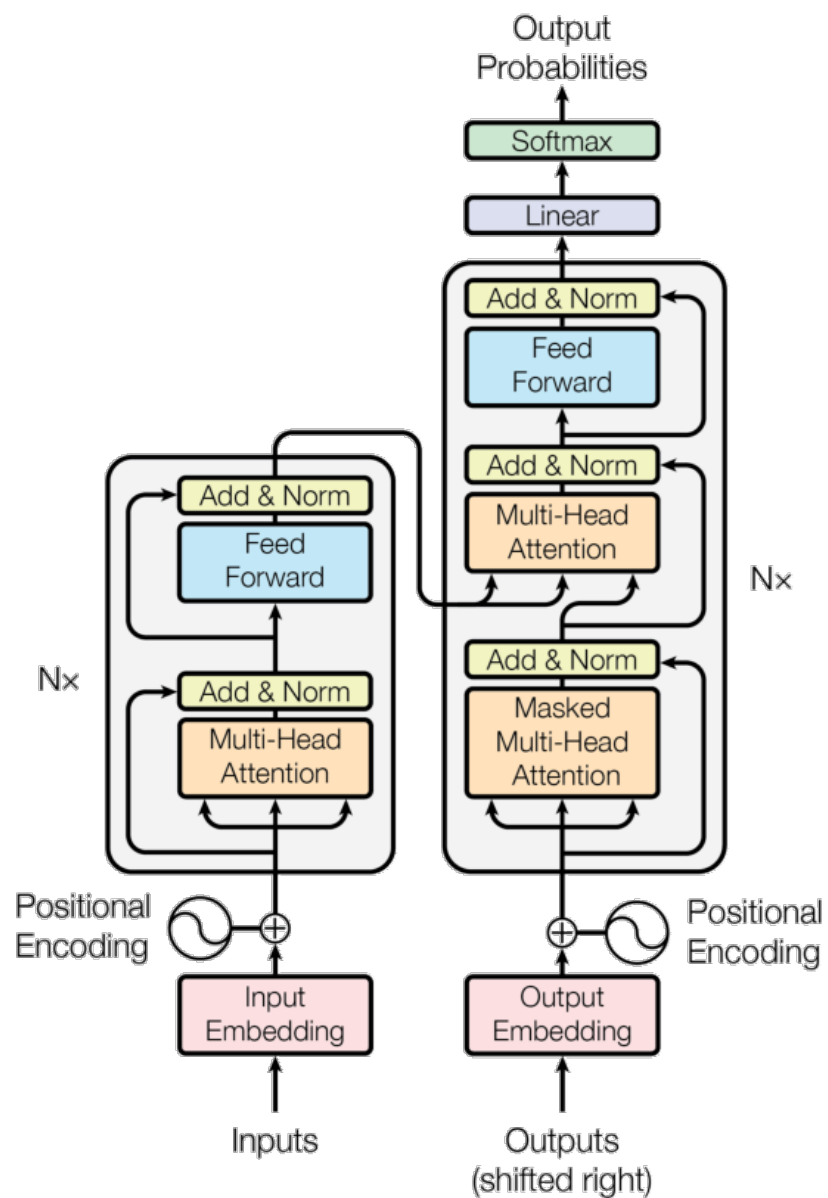


第11章 大语言模型

复习: Transformer

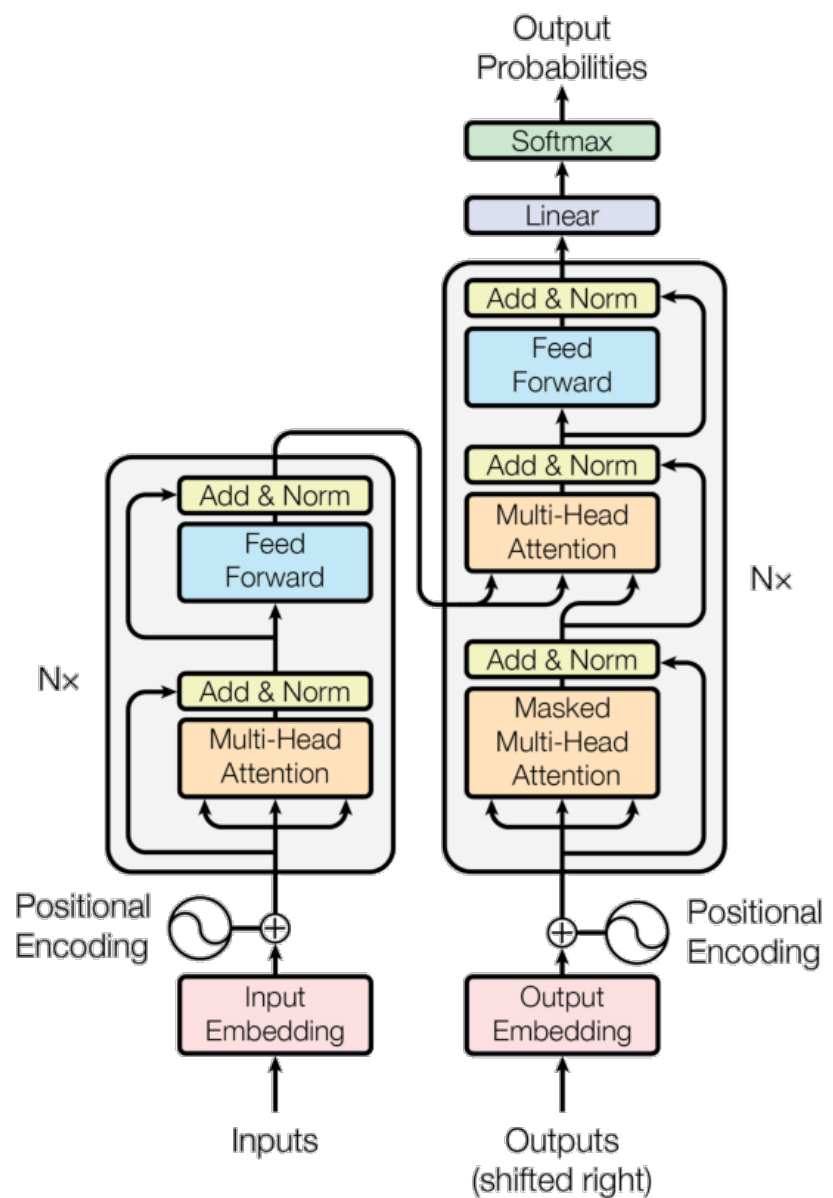


Transformer模型架构



- Transformer的**输入**首先是一串经过分词的**token序列**。
- Embedding层会把这些token转换成可计算的**向量表示**。
- 同时通过Positional Encoding注入序列的**位置信息**，让模型知道词语的顺序。
- 得到的向量序列会被送入N层Encoder，逐层**提取**并压缩**语义特征**。
- 随后，N层Decoder会利用Encoder提供的语义表示，一步步**生成目标序列**。
- 最后，Decoder的**输出**会经过线性层 + Softmax，得到每个词的**概率分布**，从而预测最终的结果。

Transformer模型架构



- Embedding是如何实现的?
- 什么是位置编码? 其作用是什么?
- 为什么需要多头注意力?
- 什么是层归一化? 其作用是什么?
- Transformer的Feed Forward模块是什么? 有什么特点? 其作用是什么?
- 为什么解码器的多头注意力模块需要Masked?
- 什么是交叉注意力?
-

预训练语言模型 (PLM)



PLM分类

- **Encoder-only**
- **Encoder-Decoder**
- **Decoder-only**

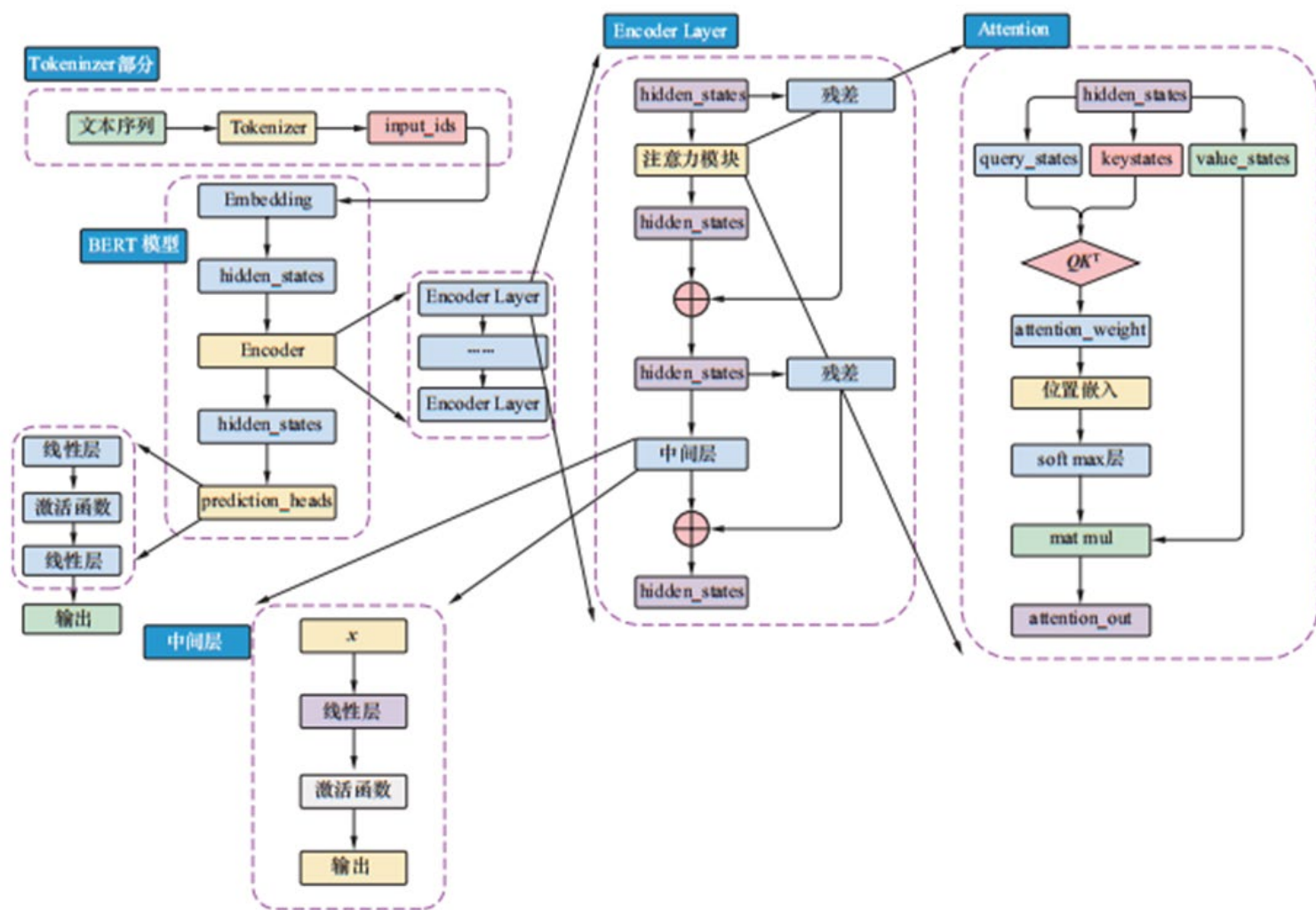
Encoder-only PLM

- 是指仅由Transformer Encoder模块堆叠构建的预训练语言模型。
- 预训练任务
 - 通过随机遮盖输入文本中的部分token并预测还原，学习上下文双向表征；
 - 常辅以NSP（下一句预测）或SOP（句序预测）等句间关系建模任务，强化长程语义理解能力。
- 擅长的任务领域
 - 天然适配自然语言理解（NLU）场景；
 - 典型任务包括文本分类（情感分析、意图识别）、语义匹配（相似性判断、推理）、信息抽取（实体/关系提取）及阅读理解（答案定位、逻辑推理）。
- 代表性模型
 - BERT
 - BERT的改进形式，如RoBERTa、ALBERT等

Encoder-only PLM

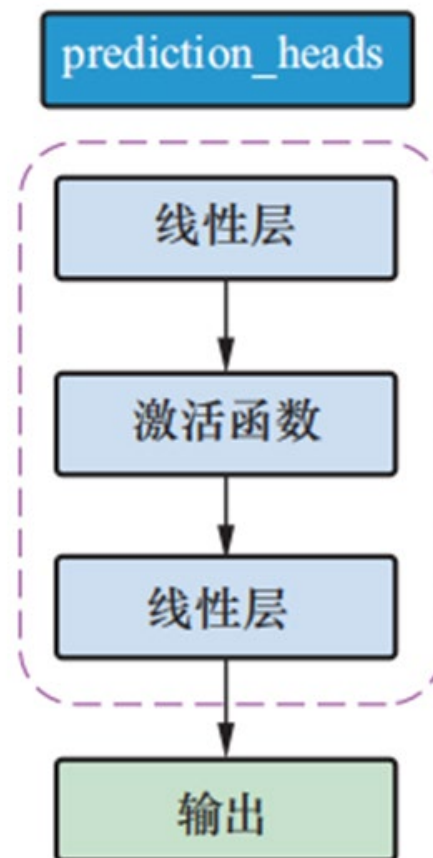
● BERT

- BERT模型架构取Transformer的Encoder部分堆叠而成，其主要结构如右图所示。
- 核心模块：
 - Tokenizer
 - Embedding
 - Encoder (N层堆叠)
 - prediction_heads



● BERT的Prediction Heads

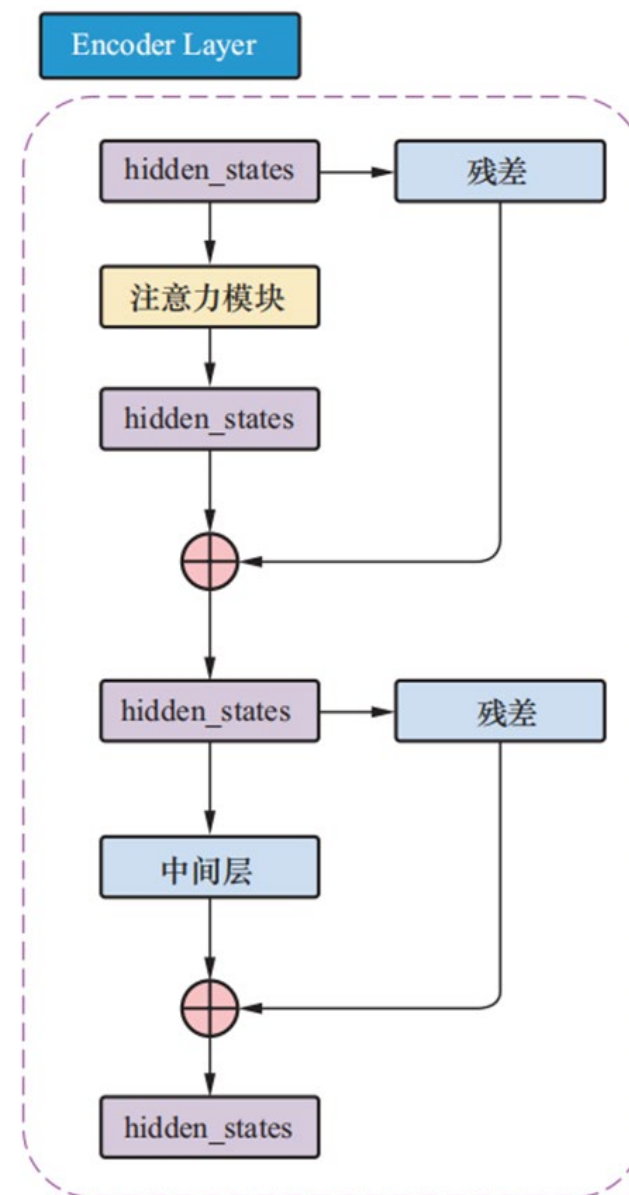
- Encoder输出的隐藏状态送入prediction heads进行任务相关预测。具体架构如下：
- 第一层 线性层
 - 输入：Encoder 输出的隐藏向量。
 - 作用：对特征进行重新组合和映射。
- 激活函数
 - 常用：GELU。 $\text{GELU}(x) = 0.5x \left(1 + \tanh \left(\sqrt{2/\pi}(x + 0.044715x^3) \right) \right)$
 - 作用：增强模型表达能力，使模型能学习更复杂关系。
- 第二层 线性层
 - 将非线性特征映射到目标输出空间。



Encoder-only PLM

● BERT的Encoder Layer

- 每一层Encoder Layer都是和Transformer中的 Encoder Layer结构类似的层，具体结构如图所示：
- Attention：已经通过 Embedding 层映射的 hidden_states进入了核心的注意力模块。
- 残差连接：通过残差连接机制和原输入相加。
- 中间层：经过中间层便能够得到最终的输出。



● BERT的中间层

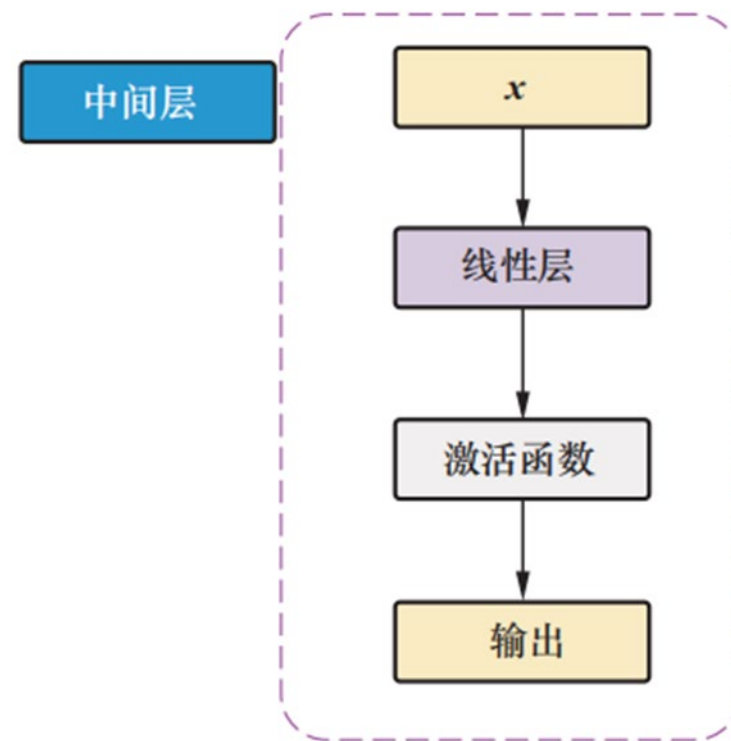
- Intermediate 层就是“线性层 + 激活函数”，这是为了让模型学到更复杂的特征表示，具体流程是：
- 输入：输入向量 x 来自上一层。
- 线性层：对输入做线性变换，把特征从一个空间映射到另一个空间

$$z = Wx + b$$

- 激活函数 (GELU)

$$GELU(x) = 0.5x(1 + \tanh(\frac{\sqrt{2}}{\pi})(x + 0.044715x^3))$$

- 输出结果：得到新的输出，用于后续网络计算。

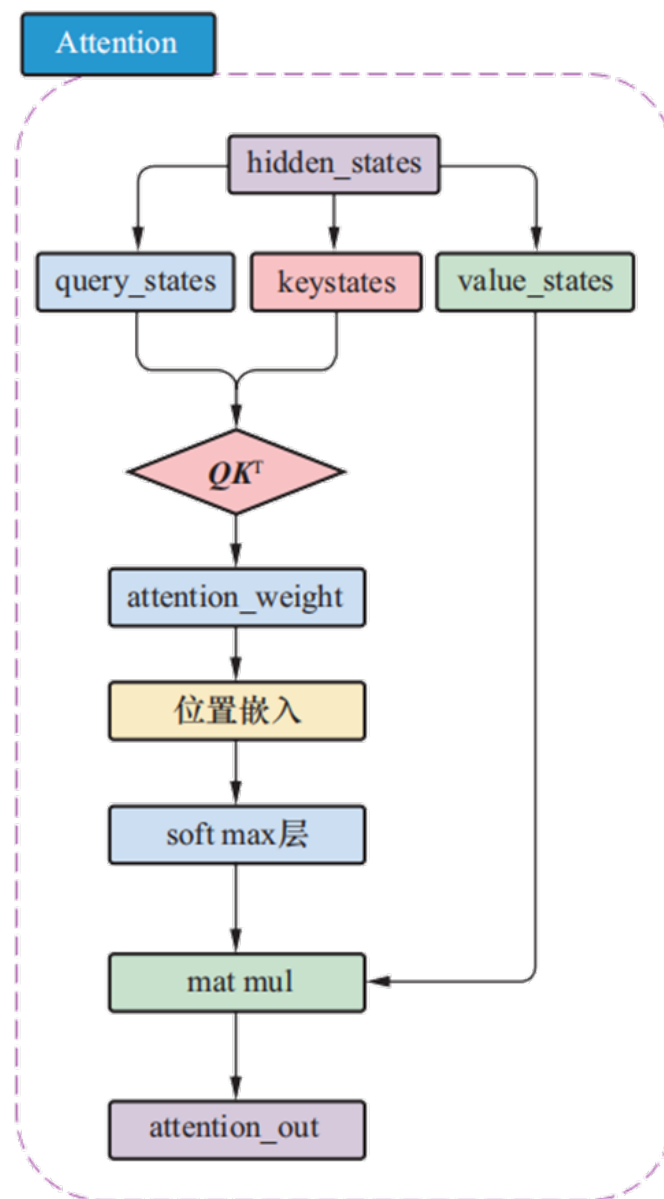


● BERT的Attention模块

- BERT 的注意力机制和 Transformer 中Encoder 的自注意力机制几乎完全一致，但是 BERT 不仅将相对位置编码融合到了注意力机制中，还将它们视为可训练的权重参数，具体流程：
- 生成 Q / K / V：由同一输入隐藏状态得到三组向量：

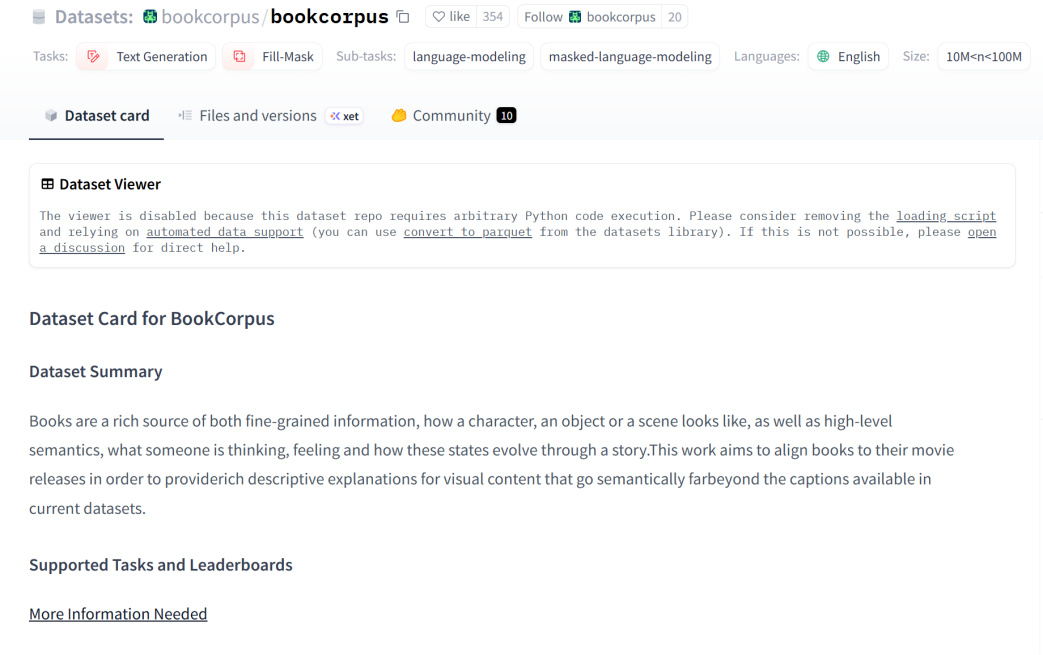
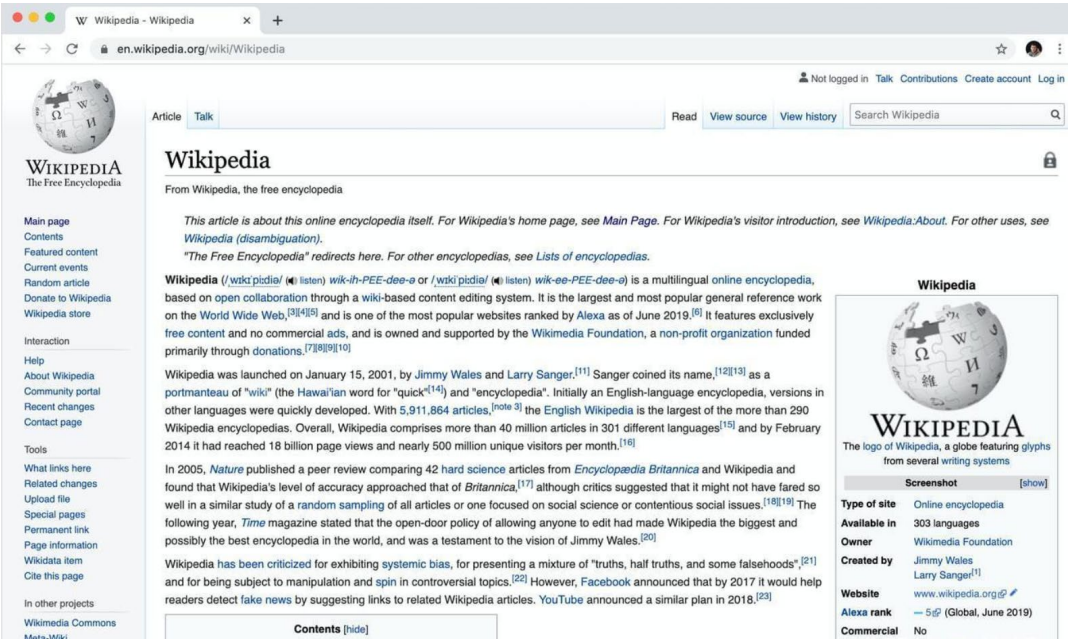
$$Q = HW_Q, K = HW_K, V = HW_V$$

- 计算注意力分数：通过 Query 与 Key 的相似度衡量关联强度。
- 位置嵌入：这里的 Position Embedding 层其实就是一层线性矩阵。



Encoder-only PLM

- BERT的自监督训练任务：
 - BERT的预训练数据主要由英文维基百科（English Wikipedia） 和BookCorpus两大语料库构成，数据总量共约16G， 约33亿单词



Encoder-only PLM

● BERT的自监督训练任务:

- MLM: 在一个文本序列中随机遮蔽部分token, 然后将所有未被遮蔽的token输入模型, 要求模型根据输入预测被遮蔽的token

输入: I <MASK> you because you are <MASK>
输出: <MASK> - love; <MASK> - wonderful

- NSP: 针对句级的NLU任务, 例如问答匹配、自然语言推理等, 核心思想是要求模型判断一个句对的两个句子是否为连续的上下文

输入:
Sentence A: I love you.
Sentence B: Because you are wonderful.
输出:
1 (正样本)

输入:
Sentence A: Because you are wonderful.
Sentence B: I love you.
输出:
0 (负样本)

Encoder-Decoder PLM

- **Encoder-Only模型的特点：**

- 擅长理解类（NLU）任务并在下游微调取得好成绩。
- 但预训练目标（如 MLM/NSP）与不少下游任务存在目标不一致，且对生成式任务与更灵活输入输出形式支持有限。

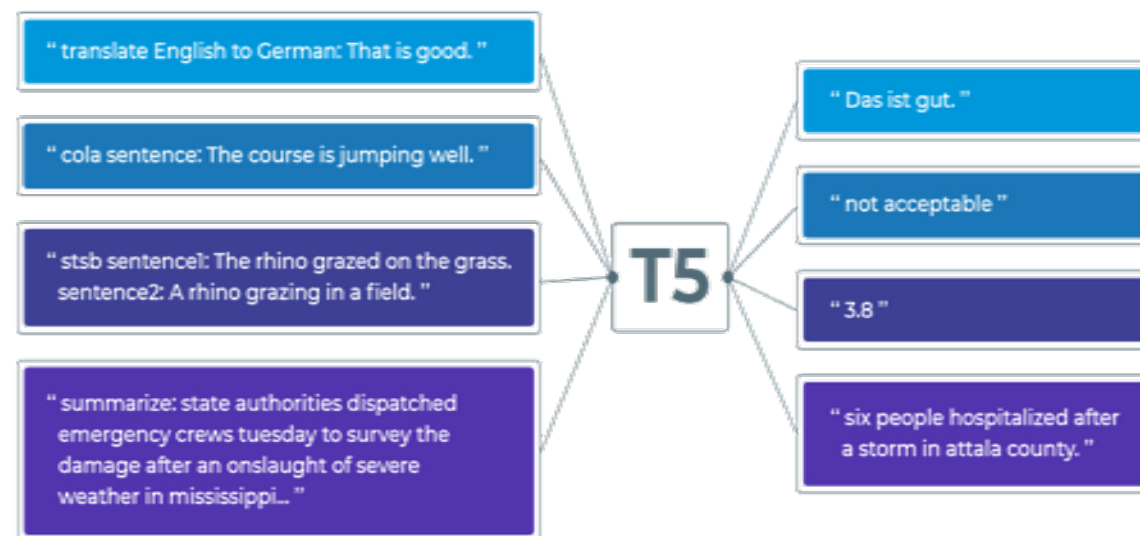
- **Encoder-Decoder模型：**

- 引入Encoder-Decoder结构以实现序列到序列的端到端建模，为翻译、摘要、问答等生成与理解一体化任务提供统一范式。
- 以T5模型为代表。

Encoder-Decoder PLM

● T5的特点:

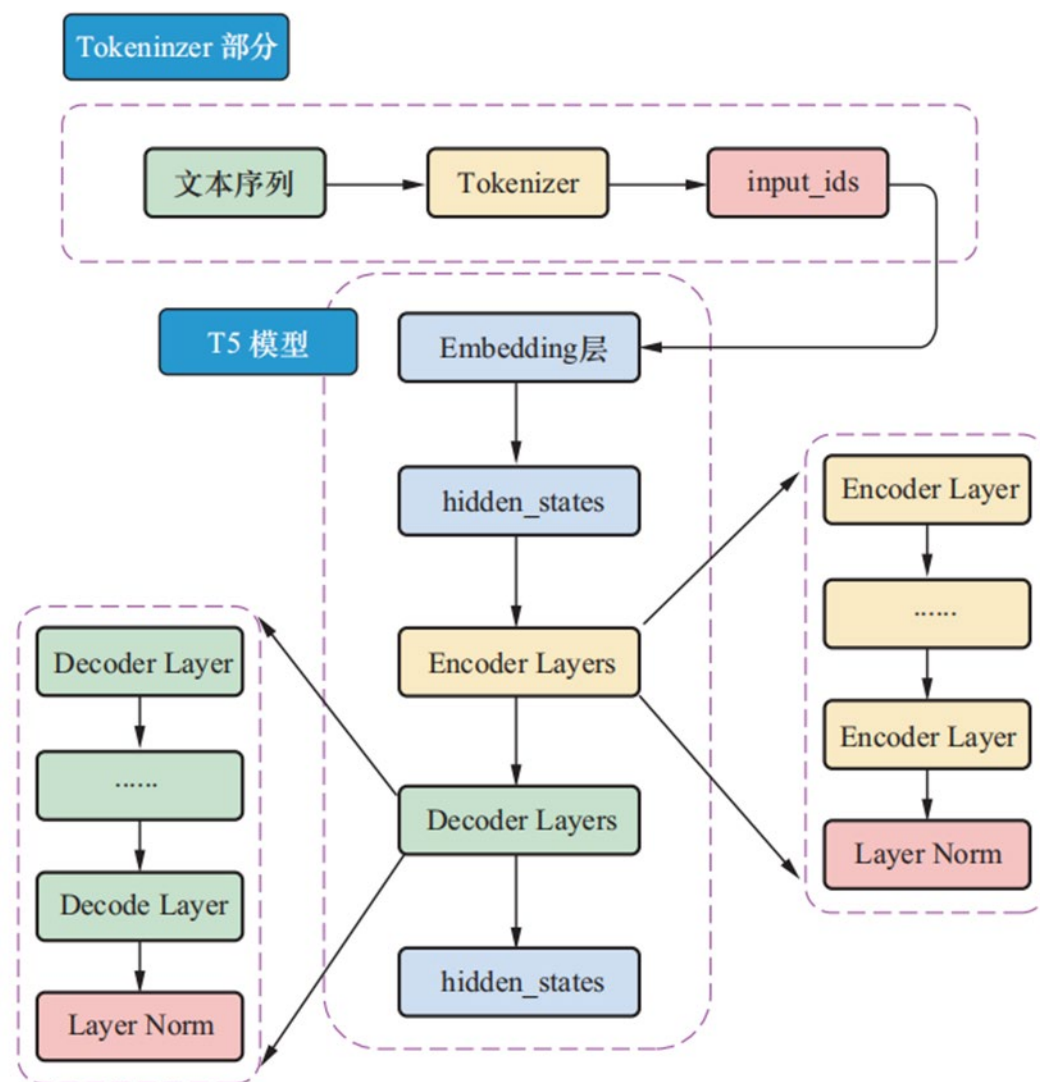
- 所有NLP任务转化为“文本→文本”形式，使用同一套模型、损失函数和超参数处理翻译、摘要、问答等任务。
- 基于Transformer架构，包含编码器和解码器两个部分。
- 利用相对位置编码处理长序列中的位置信息，并在每层中包含前馈神经网络以进一步处理特征。
- 减少了任务特定的模型调试工作，还能够使用相同的数据处理和训练框架。



Encoder-Decoder PLM

● T5模型的整体架构

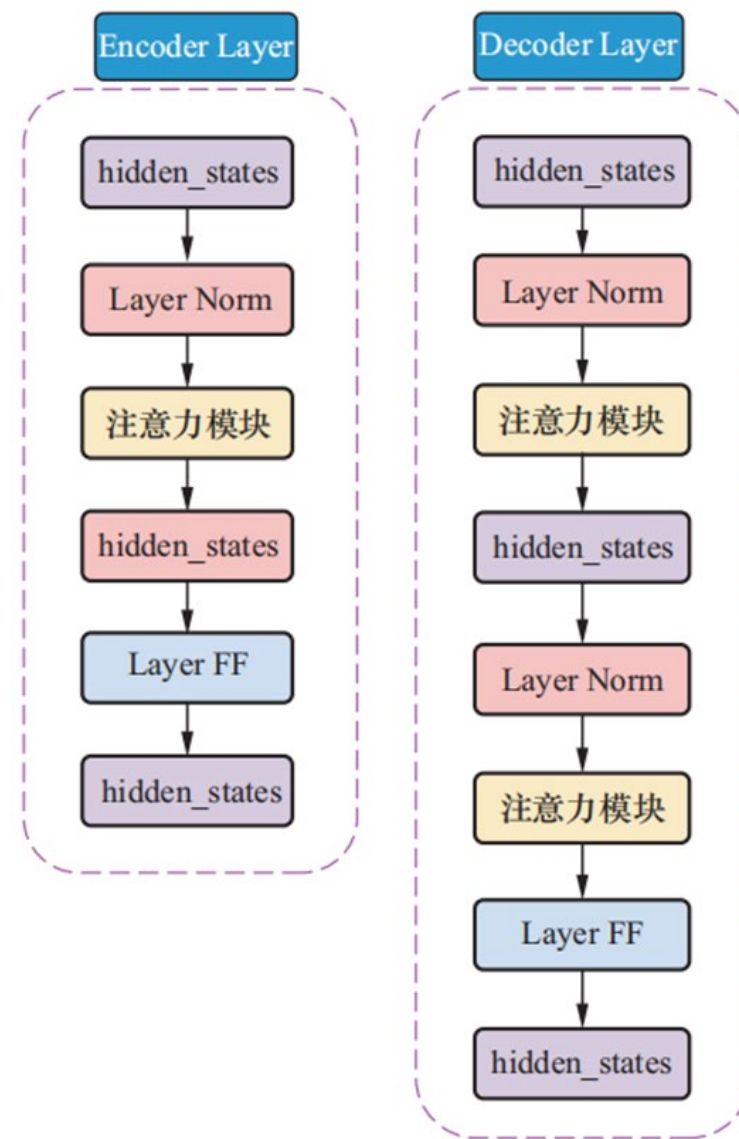
- T5的模型架构包括两部分，分别为Tokenizer部分和Transformer部分（即 Encoder-Decoder部分）。整体结构如右图：
- Tokenizer部分：主要负责将输入文本转换为模型允许的输入格式，包括分词、编码等操作。
- Transformer部分：Transformer 部分又分为 Encoder Layers和Decoder Layers两部分，它们分别由一个个小的模块组成。
- 模块组成：每个模块包含了多头注意力机制、前馈神经网络和Layer Norm。



Encoder-Decoder PLM

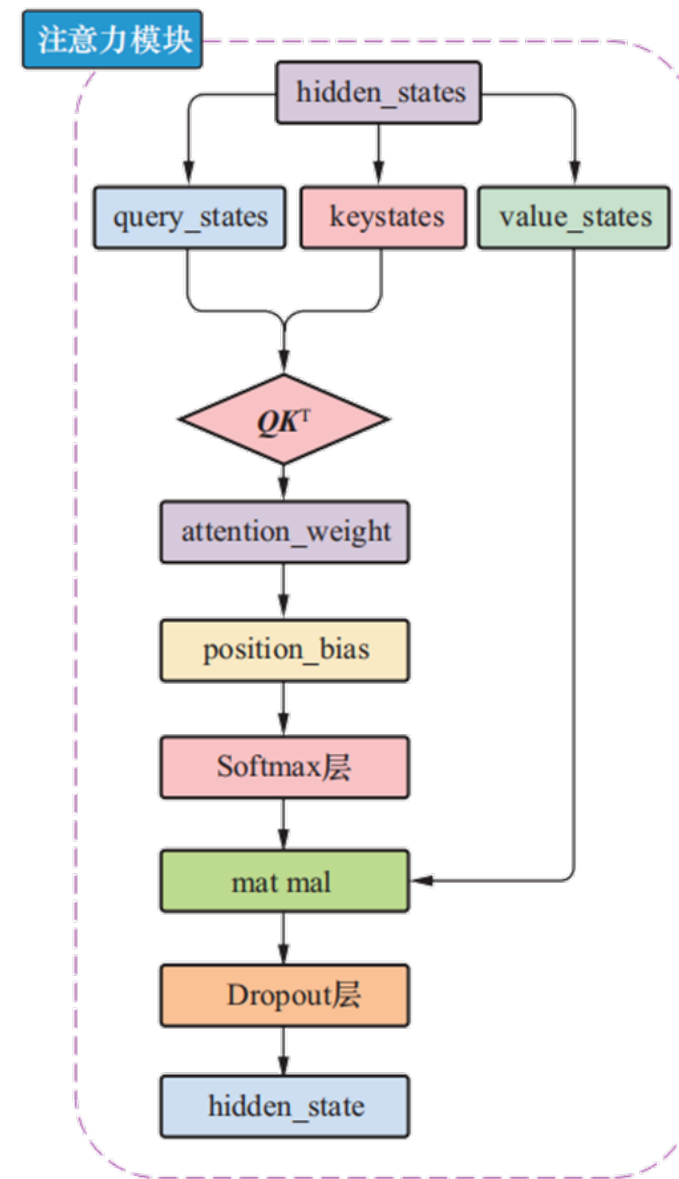
• T5的Encoder和Decoder

- T5的Encoder和Decoder部分都是基于 Transformer 架构设计的，主要包括自注意力和前馈神经网络两种结构。架构如右图所示。
- 自注意力用于捕捉输入序列中的全局依赖关系，前馈神经网络用于处理特征的非线性变换。
- Decoder中还包含了Encoder-Decoder Attention结构，用于捕捉输入和输出序列之间的依赖关系。这两种注意力结构几乎完全一致，仅在位置编码和 Mask 机制上有所不同。



● T5的Attention机制

- T5的Attention机制和BERT的Attention机制是一样的，都是基于Self-Attention机制设计的。
- 由隐藏状态生成 Q / K / V：输入 hidden_states 分别线性映射。
- 计算注意力权重：通过计算 Query、Key和 Value 之间的相似度来捕捉输入序列中的全局依赖关系。
- 与Mask机制对比：Encoder-Decoder Attention仅仅在位置编码和Mask机制上有所不同，主要是为了区分输入和输出序列。



● RMSNorm

- 与原始Transformer模型不同，T5模型的LayerNorm采用了RMSNorm，通过计算每个神经元的均方根（Root Mean Square）来归一化每个隐藏层的激活值。
- RMSNorm的参数设置与LayerNormalization相比更简单，只有一个可调参数，可以更好地适应不同的任务和数据集。RMSNorm函数可以用以下数学公式表示：

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\sqrt{\frac{1}{n} \sum_{i=1}^n w_i^2 + \epsilon}}$$

- 目标和LayerNorm类似：稳定训练、控制激活尺度、改善梯度传播。
- 但把LayerNorm的“均值中心化”步骤去掉了，只用 RMS（均方根）来做缩放，因此更省算、更简单，常见于很多现代大模型结构中。

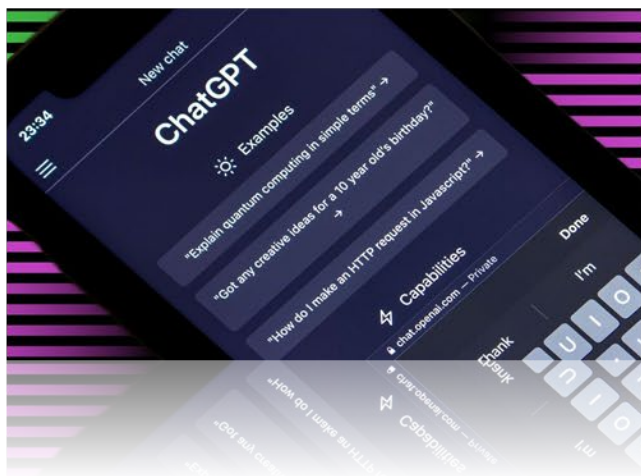
● T5的自监督训练任务：

- **预训练任务：** T5模型的预训练任务是MLM，也称为BERT-style目标。具体来说，就是在输入文本中随机遮蔽15%的token，然后让模型预测这些被遮蔽的token。这个过程不需要标签，可以在大量未标注的文本上进行。
- **输入格式：** 预训练时，T5将输入文本转换为“文本到文本”的格式。对于一个给定的文本序列，随机选择一些token进行遮蔽，并用特殊的占位符替换。然后将被遮蔽token序列作为模型的输出目标。
- **预训练数据集：** T5使用了自己创建的大规模数据集Colossal Clean Crawled Corpus (C4)，该数据集从Common Crawl中提取了大量干净的英语文本。C4数据集经过了一定的清洗，去除了无意义的文本、重复文本等。
- **多任务预训练：** T5还尝试了将多个任务混合在一起进行预训练，而不仅仅是单独的MLM任务。这有助于模型学习更通用的语言表示。

Decoder-Only PLM

- 以GPT系列模型为代表：

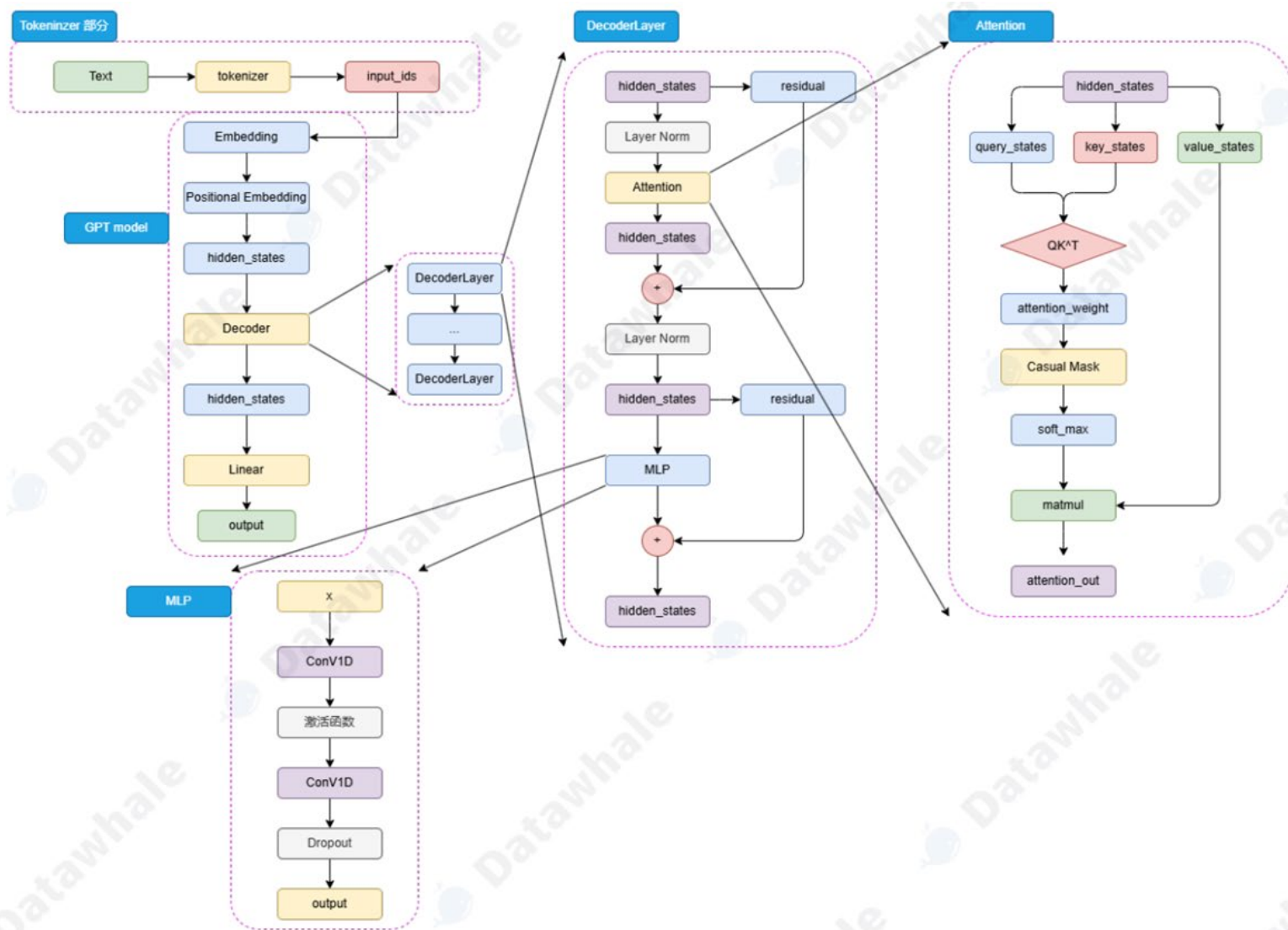
- 由OpenAI团队于2018年发布
- LLM时代的最大赢家
- 核心特点是能够生成自然、流畅的语言，并具备回答问题、生成文本、语言翻译等多种功能



Decoder-Only PLM

● 核心流程:

1. 将输入文本进行分词和嵌入，并加入位置编码来表示语序；
2. 这些信息会流经一叠相同的解码器层 (Decoder Layer)；在每个解码器层内部，核心是带掩码的自注意力机制，它确保模型在预测下一个词时只能看到前面的内容；
3. 通过一个前馈网络 (MLP) 进行信息处理；通过一个线性层 (Linear) 将最终结果映射到词汇表，生成输出。



Decoder-Only PLM

● GPT自监督预训练任务——CLM

- 采用因果语言模型（CLM）进行预训练，其本质就是一个“文本接龙”或“自动补全”任务：根据所有已经出现的上文内容，来预测下一个最可能的词，并不断重复这个过程，最终生成完整的文本序列。

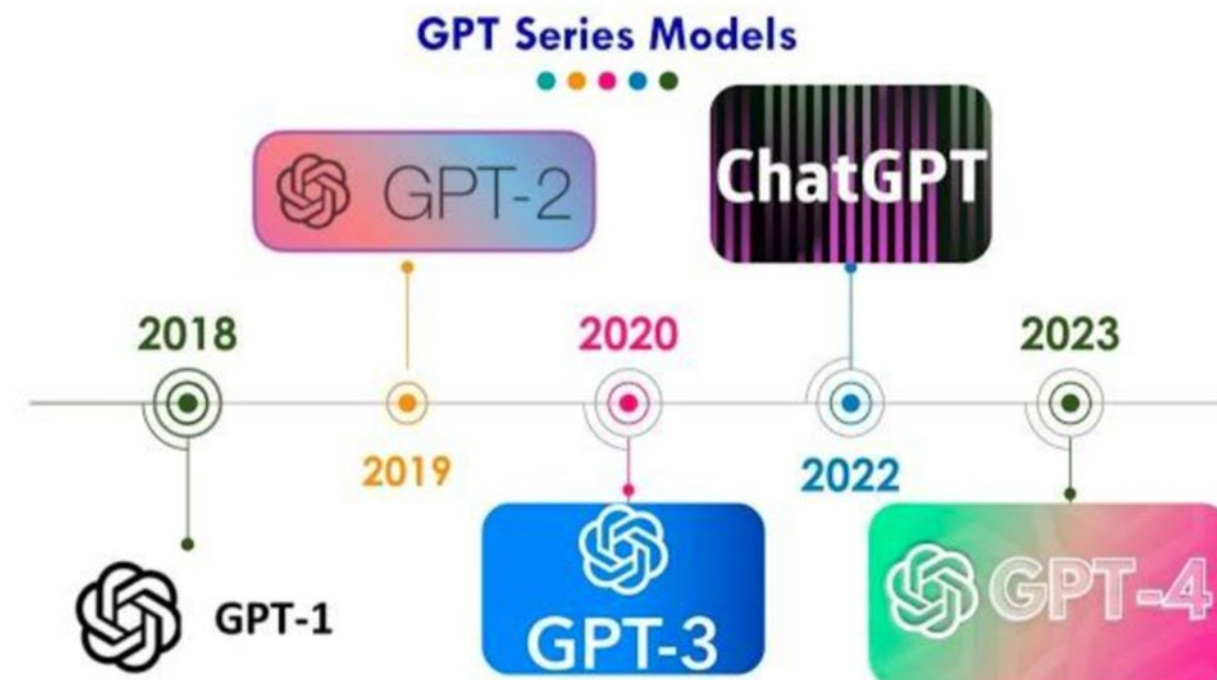
input: 今天天 output: 今天天很
input: 今天天很 output: 今天天很好

- 任务目标：预测下一个词 (Next-token Prediction)。
- 输入：一段已有的文本序列（例如：“今天天气”）。
- 输出：在原有序列后追加一个新预测的词（例如：“今天天气很”）。
- 生成方式：将新的、更长的序列作为下一次的输入，循环往复，逐词生成完整句子。

Decoder-Only PLM

● GPT系列模型的发展

- GPT-1：首个采用纯解码器架构的预训练模型，为系列发展奠定了技术路线
- GPT-2：将模型规模提升十倍，首次向世界展示了惊人的“零样本”学习能力，证明了仅靠扩大规模就能让模型在不微调的情况下完成多种新任务。
- GPT-3：参数规模达到1750亿，带来了革命性的“上下文学习”（In-context Learning）能力，仅需在提示中提供少量示例便能高效完成新任务，从而正式引爆了大模型时代。

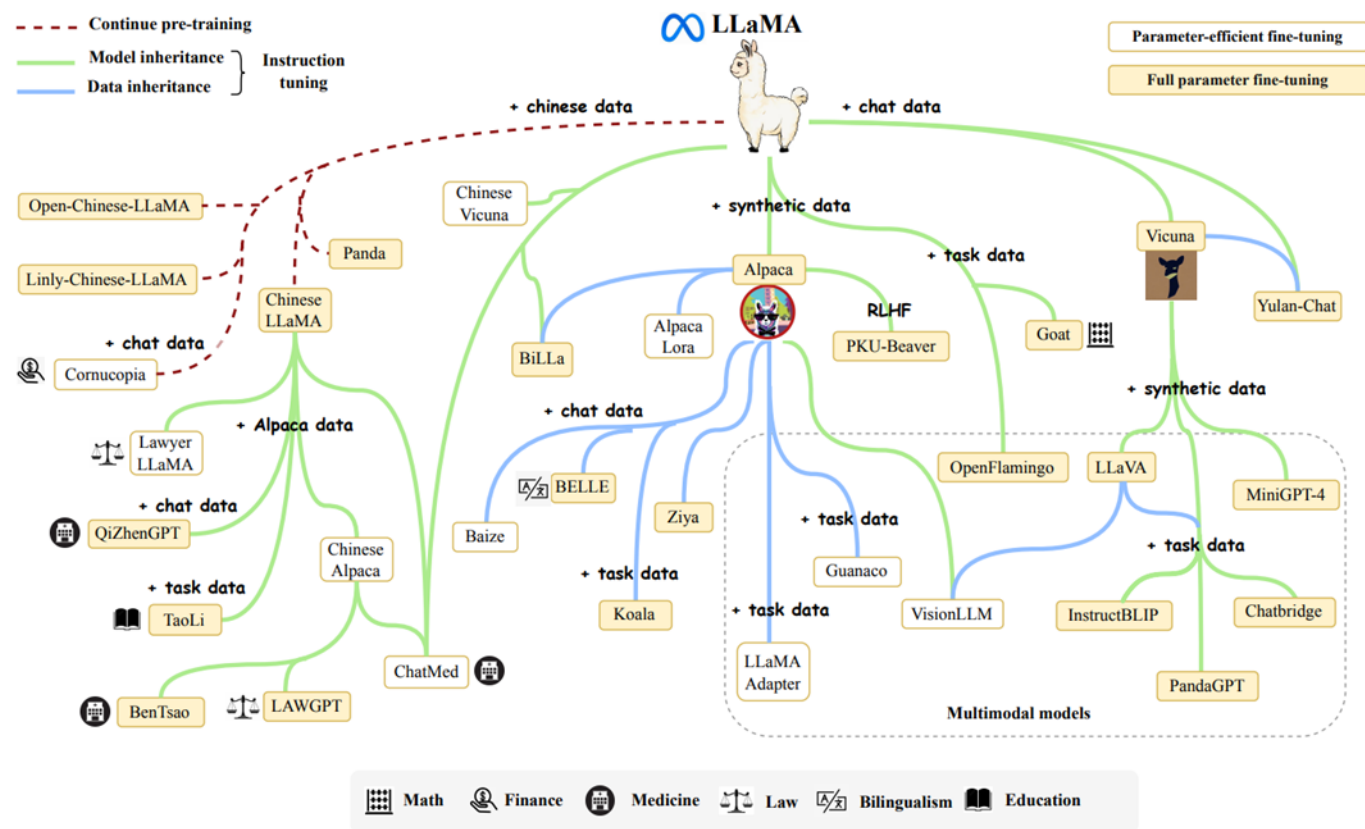


模型	Decoder Layer	Hidden_size	注意力头数	注意力维度	总参数量	预训练语料
GPT-1	12	3072	12	768	0.12B	5GB
GPT-2	48	6400	25	1600	1.5B	40GB
GPT-3	96	49152	96	12288	175B	570GB

Decoder-Only PLM

● LLaMA:

- Meta发布的开源LLM
- LLaMA-1 (2023) 使用超过 1T token 的语料进行训练。
- LLaMA-2 (2023) 训练语料翻倍至 2T token, 上下文长度也从2K翻倍到 4K。同时引入了分组查询注意力 (GQA) 等新技术以提升效率。
- LLaMA-3 (2024) 训练语料提升到 15T token, 支持8K长上下文, 并采用了效率更高的Tokenizer。



Decoder-Only PLM

• GLM

- 由智谱AI (Zhipu AI) 与清华大学联合推出的一个开源大语言模型系列
- 核心创新在于其独特的预训练任务，结合了BERT的MLM思想与GPT的CLM思想。具体来说，GLM并非像BERT那样预测单个被遮盖的词，而是遮盖一整段连续的文本 (span)，然后要求模型像GPT一样自回归地、逐词生成被遮盖的整段内容。

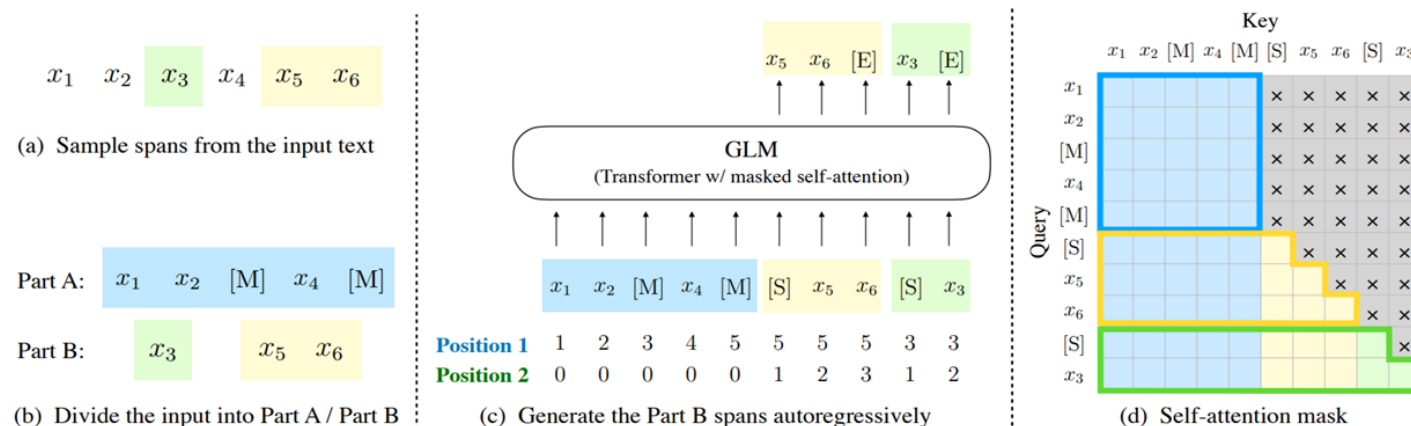
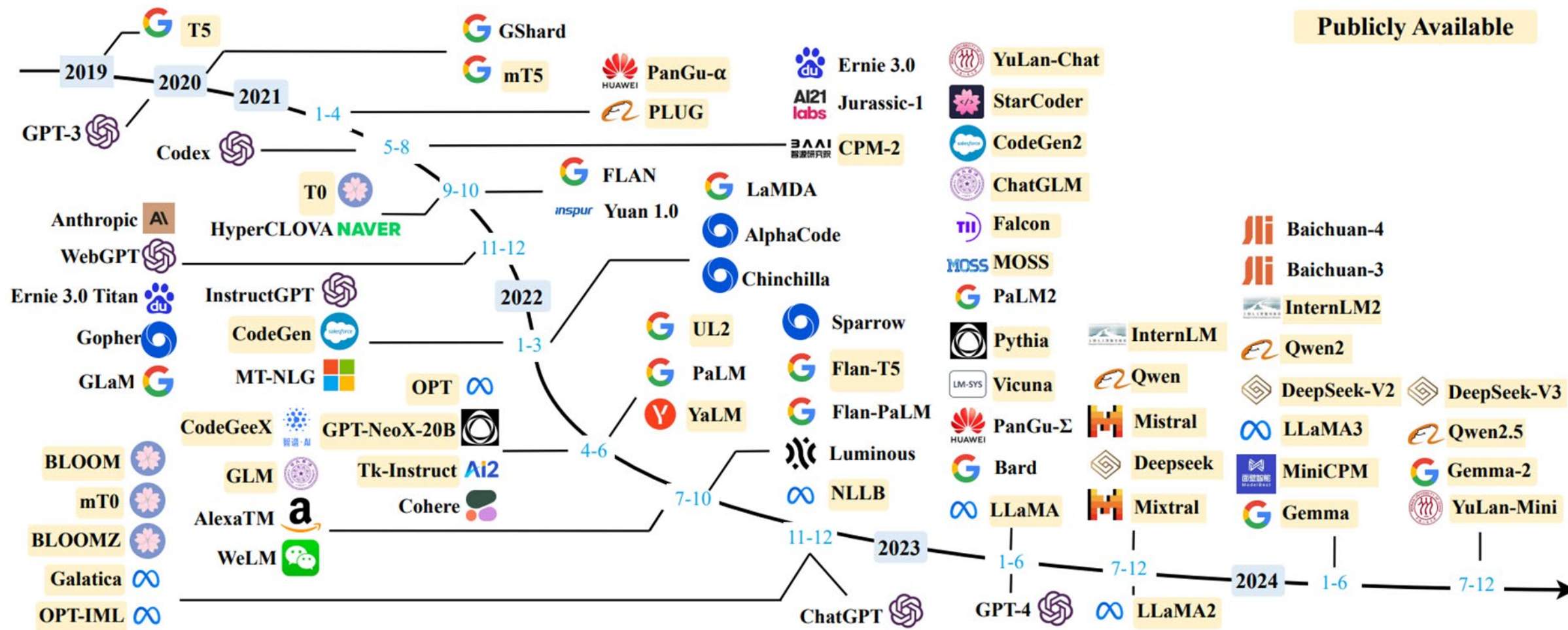


Figure 2: GLM pretraining. (a) The original text is $[x_1, x_2, x_3, x_4, x_5, x_6]$. Two spans $[x_3]$ and $[x_5, x_6]$ are sampled. (b) Replace the sampled spans with $[M]$ in Part A, and shuffle the spans in Part B. (c) GLM autoregressively generates Part B. Each span is prepended with $[S]$ as input and appended with $[E]$ as output. 2D positional encoding represents inter- and intra-span positions. (d) Self-attention mask. Grey areas are masked out. Part A tokens can attend to themselves (blue frame) but not B. Part B tokens can attend to A and their antecedents in B (yellow and green frames correspond to the two spans). $[M] := [\text{MASK}]$, $[S] := [\text{START}]$, and $[E] := [\text{END}]$.

大语言模型 (LLM)



- 2019年以来，全球大语言模型研究一日千里，模型发布的密度越来越高，参与者日益多元化，技术持续快速突破。



LLM的定义

- LLM，即Large Language Model，中文名为大语言模型或大型语言模型，是一种相较传统语言模型参数量更多、在更大规模语料上进行预训练的语言模型。
 - LLM 指包含**数百亿（或更多）参数**的语言模型
 - 它们往往在**数TB token或更多**语料上通过多卡分布式集群进行预训练，具备远超出传统预训练模型的文本理解与生成能力
 - 随着LLM研究的不断深入，多种参数尺寸的LLM逐渐丰富，广义的LLM覆盖了从十亿参数（如 Qwen-1.5B）到千亿参数（如 Grok-314B）的所有大型语言模型
 - 只要模型展现出涌现能力，即在一系列复杂任务上表现出远超传统预训练模型（如 BERT、T5）的能力与潜力，都可以称之为LLM

LLM的常见能力

- **大模型旨在提供更强大、更准确的模型性能，以应对更复杂、更庞大的数据集或任务。**
大模型通常能够学习到更细微的模式和规律，具有更强的泛化能力和表达能力。

涌现能力

所谓“涌现能力”，就是指LLM在模型规模足够大时，会像物理学中的“相变”一样，突然获得之前没有的、解决复杂通用任务的强大能力。



上下文学习能力

允许语言模型在提供自然语言指令或多个任务示例的情况下，通过理解上下文并生成相应输出的方式来执行任务，而无需额外的训练或参数更新。



指令遵循能力

指令遵循指的是LLM能泛化并遵从自然语言指令来执行任务，即使是未曾专门训练过的任务，从而极大提升了模型的通用性和实用性。



逐步推理能力

逐步推理能力使LLM能够像人一样思考，通过“一步一步想”的方式解决复杂的逻辑问题，这让它在处理数学、编程等任务时表现出色，极大地提升了其实用价值。



LLM的基本特点

● 多语言支持

- LLM由于需要使用到海量的语料进行预训练，训练语料往往本身就是多语言的，因此天生具有多语言、跨语言能力；
- 只不过随着训练语料和指令微调的差异，在不同语言上的能力有所差异。

● 长文本处理

- LLM往往比传统PLM更看重长文本处理能力。相对于以512 token为惯例的传统，LLM往往在训练时就支持4k、8k甚至32k的上下文长度；
- LLM大部分采用了旋转位置编码（RoPE）作为位置编码，具有一定的长度外推能力，也就是在推理时能够处理显著长于训练长度的文本。



LLM的基本特点

● 拓展多模态

- 随着LLM的不断改进，通过为LLM增加额外的参数来进行图像表示，从而利用LLM的强大能力构造支持文字、图像双模态的模型，已经是一个成功的方法；
- 通过引入Adapter层和图像编码器，并针对性地在图文数据上进行有监督微调，模型能够具备不错的图文问答甚至生成能力。

● 极强的通用性

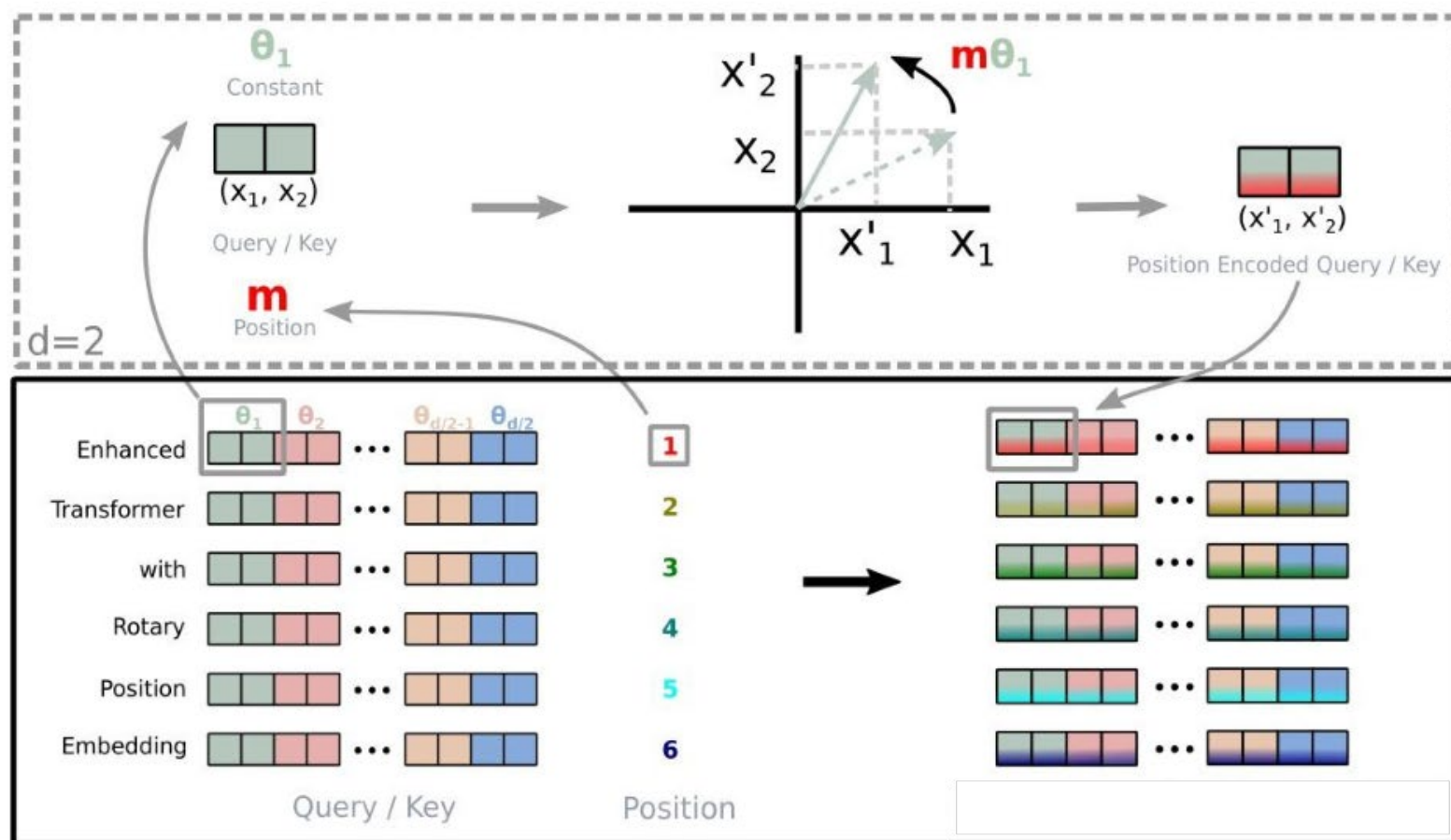
- 不再是“专才”（如以前的翻译专用模型、分类专用模型），而是“通才”。同一个模型可以写代码、写诗、做翻译、做摘要、扮演角色等。



旋转位置编码 (RoPE)

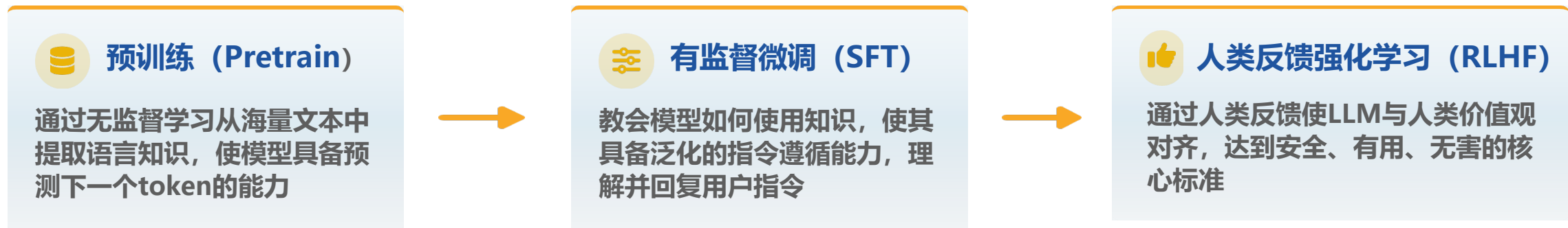
● 旋转位置编码

- 旋转嵌入是LLaMA等模型中的重要组件，为注意力机制提供位置上下文信息
- 通过将位置信息编码为旋转矩阵的形式，使模型能够利用相对位置关系增强注意力计算



如何训练LLM

- LLM训练是一个层层递进的过程：



- 每个阶段都赋予模型特定的能力和特性，最终使其具备强大的语言理解与生成能力：

- 预训练赋予LLM海量知识；
- SFT使模型学会理解并回复用户指令，具备实用能力；
- RLHF从更深层次实现与人类价值观的对齐，确保安全性。

预训练核心概念



问题定义与目标

通过无监督学习从海量文本数据中提取语言知识，使模型具备预测下一个token的能力



架构选择

当前主流LLM（如LLaMA架构）普遍采用Decoder-Only架构，沿用GPT模型的经典预训练任务

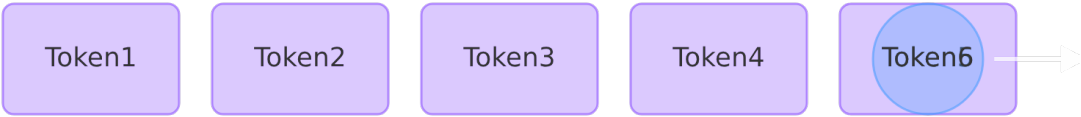


预训练任务

因果语言模型（Causal Language Model, CLM），通过给定上文预测下一个token进行训练

因果语言模型（CLM）工作原理

预测下一个token



模型通过上文预测下一个最可能的词或符号，学习语言模式和上下文关系

预训练阶段获得的能力



语言模式理解



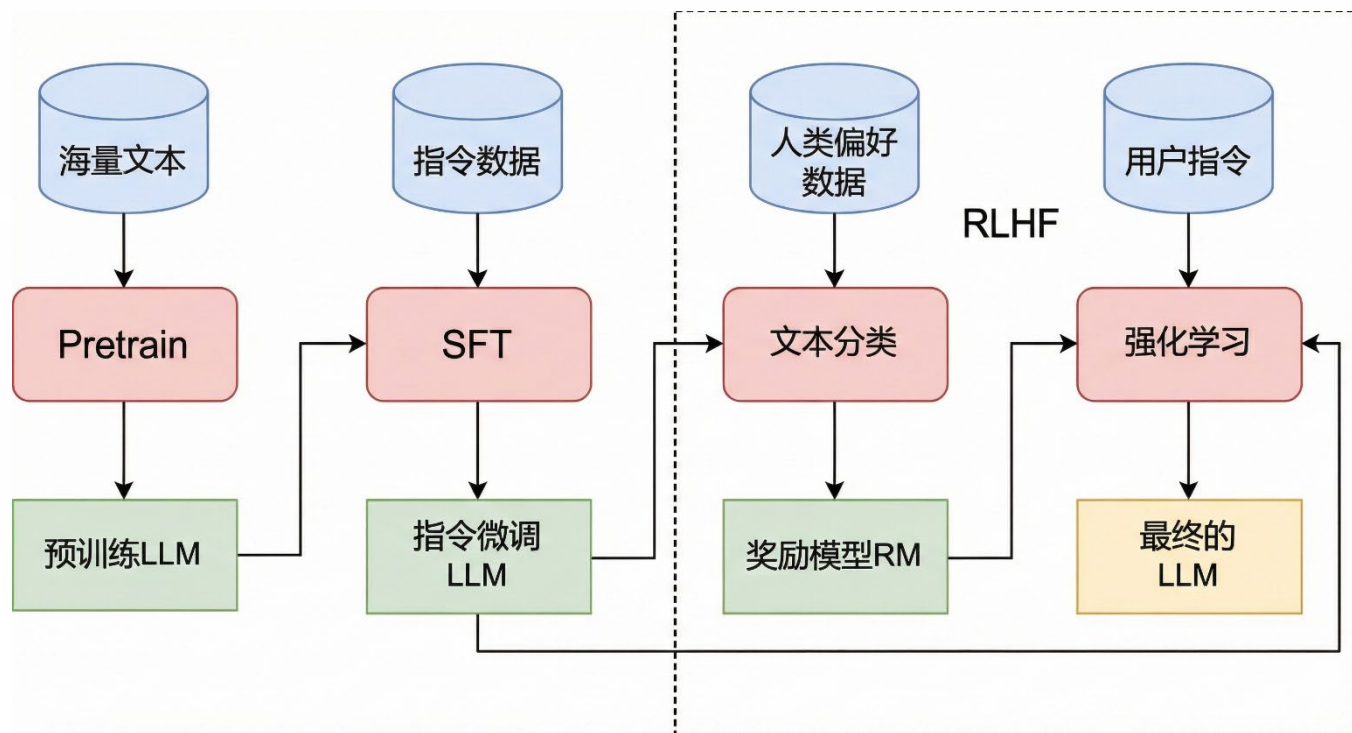
世界知识储备



连贯文本生成

● 与传统预训练模型的区别

百亿级LLM需要1024张A100训练数月，十亿级LLM也需256张A100训练数天。



● 主要挑战：

■ 算力资源需求巨大

- 百亿级参数的LLM需1024张A100训练数月，十亿级参数的LLM也需256张A100训练数天

■ 依赖分布式训练框架

- 数据并行：数据切分到不同GPU，模型参数保持一致
- 模型并行：模型拆分到多个GPU，每个GPU存放不同层
- 高效分布式：张量并行、3D并行、ZeRO等技术
- 主流框架：DeepSpeed、Megatron-LM、Colossal AI

■ 数据质量问题

- 数据体量与质量：需数百GB甚至TB级语料，质量比体量更重要
- 数据来源与配比：需涵盖多种来源并以特定比例混合
- 中文语料稀缺：高质量中文语料相对稀缺
- 数据处理与清洗：URL过滤、文档提取、语言选择、去重等

- 有监督微调（SFT）的目标是教会LLM如何使用其知识，使其具备泛化的指令遵循能力，即理解并回复用户指令。
- 与传统微调的区别：
 - 传统预训练模型需要针对每个下游任务单独微调
 - LLM的SFT通常通过指令微调（Instruction Tuning）的方式，训练模型的通用指令遵循能力

传统微调

- ✗ 需要针对每个下游任务单独进行微调
- ✗ 模型需要为特定任务定制，缺乏泛化能力
- ✗ 任务特定模型，无法处理多样化的输入
- 💡 传统预训练模型需要针对每个下游任务单独微调

VS

LLM的SFT

- ✓ 通过指令微调（Instruction Tuning）训练模型的"通用指令遵循能力"
- ✓ 使模型能在多种任务上表现良好，具备泛化能力
- ✓ 学习如何使用其知识，使其具备理解并回复用户指令的能力
- ★ 表现良好的开源LLM SFT数据量一般在数B token左右

● SFT阶段面临以下关键挑战：

■ 指令数据量及覆盖范围

- 需要收集大量类别各异的用户指令和对应回复，以获得泛化的指令遵循能力。表现良好的开源LLM SFT数据量一般在数B token左右。

■ 数据配比

- 不同类型指令数据之间的配比是关键挑战。OpenAI的InstructGPT使用了多种指令类型，并有明确的配比。

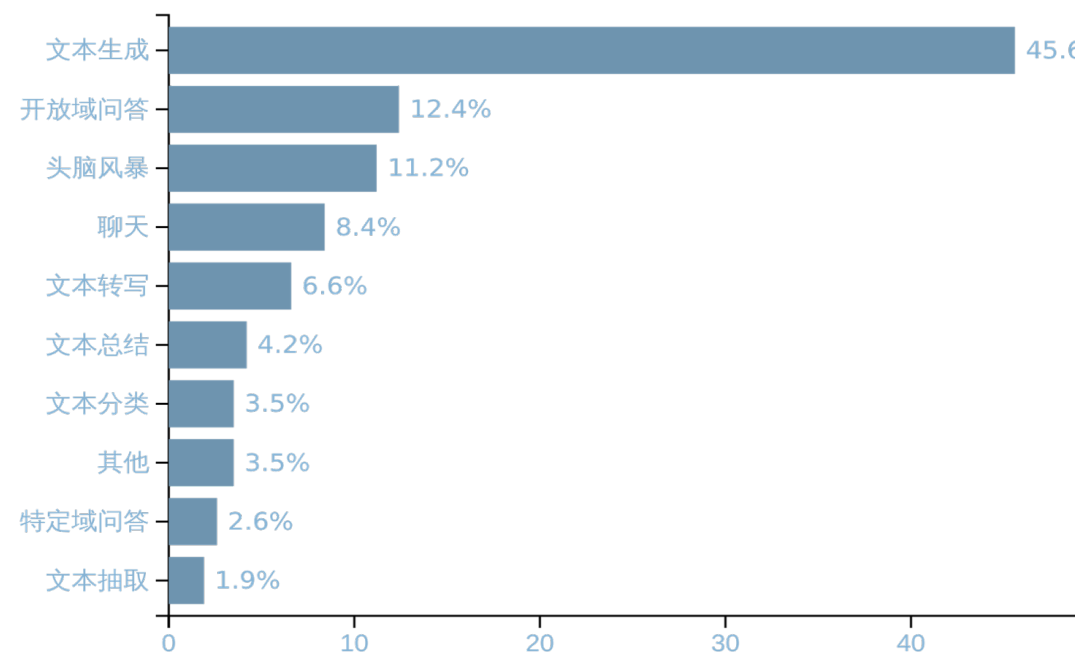
■ 高质量指令数据集获取难度

- SFT使用有监督语料，需要人工标注，成本极高。

■ 多轮对话能力

- 通过将训练数据构造成多轮对话格式，使模型能够利用历史信息生成回答。

InstructGPT指令类型及占比



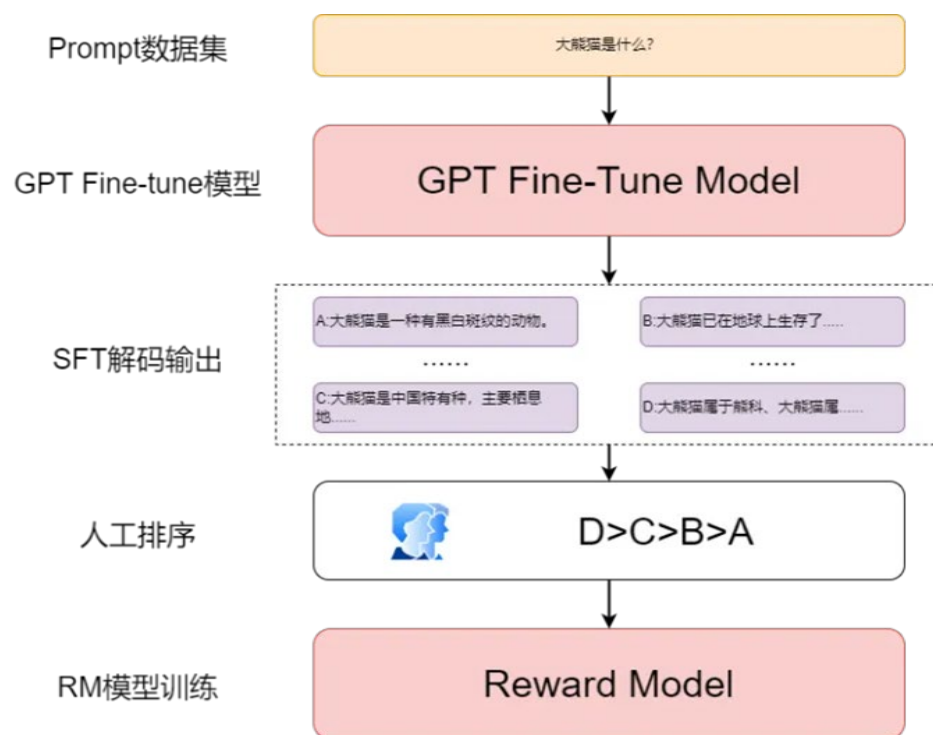
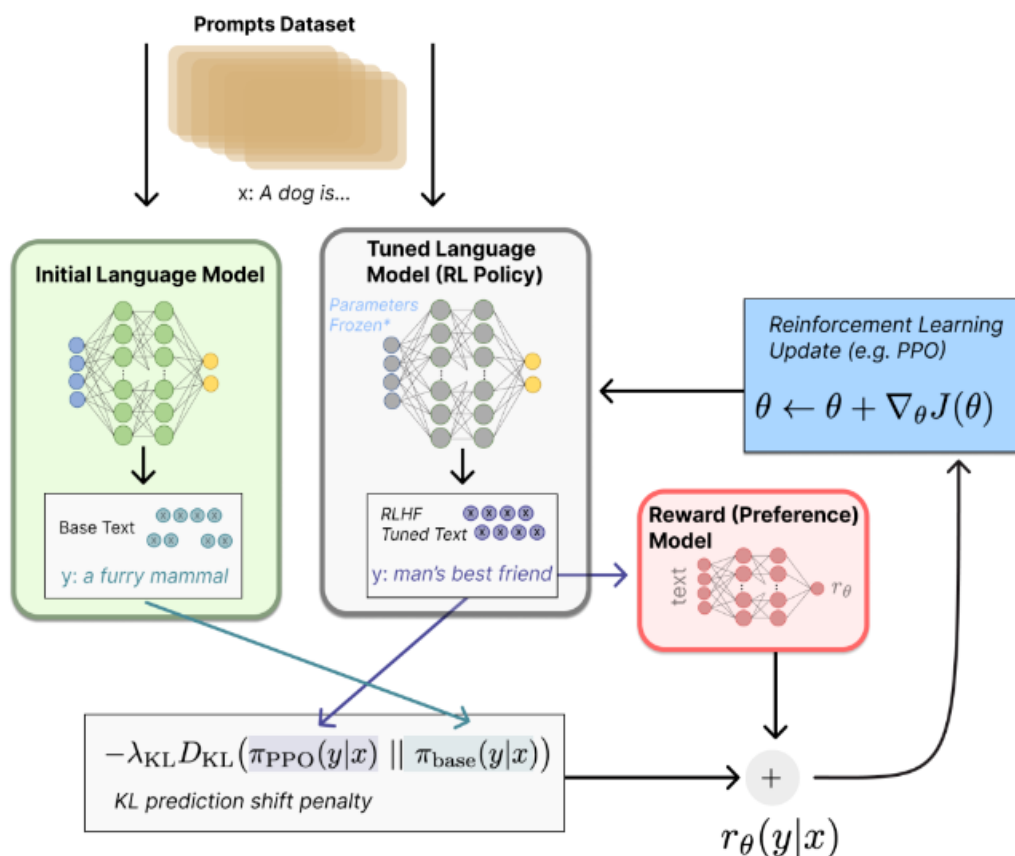
人类反馈强化学习

- **RLHF是ChatGPT相对于GPT-3的核心突破，旨在通过人类反馈使LLM与人类价值观对齐，达到安全、有用、无害的核心标准。**
- **RLHF的核心标准：**
 - 安全：确保模型生成的内容符合安全标准，不包含有害、违法或不道德的信息
 - 有用：使模型生成的内容对用户有实际帮助，准确回答问题并完成任务
 - 无害：避免模型生成可能对人类造成伤害的内容，包括偏见、歧视和不实信息

人类反馈强化学习

● 训练奖励模型 (RM) :

- 目标是拟合人类偏好，为LLM的回复打分，反映回复符合人类偏好的程度
- 本质上是一个文本分类模型，通常是传统LLM架构（或BERT架构）加上一层分类层



$$\text{loss}(\theta) = -\frac{1}{K(K-1)/2} \sum_{(x, y_w, y_l) \sim D} \log(\sigma(r_{\theta}(x, y_w) - r_{\theta}(x, y_l)))$$

人类反馈强化学习

● PPO训练

$$\mathcal{J}_{PPO}(\theta) = \mathbb{E}[q \sim P(Q), o \sim \pi_{\theta_{old}}(O|q)] \frac{1}{|o|} \sum_{t=1}^{|o|} \min \left[\frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})} A_t, \text{clip} \left(\frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})}, 1 - \varepsilon, 1 + \varepsilon \right) A_t \right]$$



Actor Model

进行微调和参数更新的LLM，从SFT后的LLM初始化



Ref Model

不进行参数更新的LLM，从SFT后的LLM初始化，限制Actor偏离



Critic Model

进行参数更新的RM，从训练好的RM初始化，输出累积奖励



Reward Model

不进行参数更新的RM，从训练好的RM初始化，输出标量奖励

● DPO训练

- Direct Preference Optimization (DPO) 是一种低门槛替代RLHF的方法，通过将强化学习问题转化为监督学习，使训练过程更简单高效。

$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right].$$

RLHF

- ✓ 需要训练奖励模型(RM)
- ✓ 需要四个模型参与训练
- ✓ 使用复杂的PPO算法
- ✓ 训练过程复杂

DPO

- ✓ 直接学习人类偏好
- ✓ 仅需两个LLM
- ✓ 无需奖励模型
- ✓ 过程更简单高效

小结

- 相较于传统NLP模型，LLM具有哪些独特的能力？
- LLM与传统预训练模型（如BERT、T5）最核心的区别是什么？
- 旋转位置编码（RoPE）技术为LLM带来了什么显著优势？
- LLM的训练大致可以分为哪三个阶段？
 - 三个阶段是什么？
 - 这三个阶段的作用分别是什么？

Tokenizer



什么是Tokenizer

- **Tokenizer（分词器）是文本与模型的桥梁，负责将自然语言转换为模型可识别的token序列，在NLP任务中被广泛应用**
- **关键作用：**
 - 文本规范化：统一文本格式，提升模型鲁棒性
 - 词汇映射：建立token与ID（即词表中的唯一编号）的对应关系
 - 上下文适配：通过特殊token标记对话角色与边界
- **分类：**
 - 基于单词（Word-based）
 - 基于字符（Character-based）
 - 基于子词（Subword-based）

Tokenizer的分类

● 基于单词 (Word-based)

- 优点：实现简单、符合人类语言直觉
- 缺点：无法处理未登录词 (OOV)、对复合词、缩写词的分割效果差
- 示例：
 - 输入: "Hello, world! There is Datawhale."
 - 输出: ["Hello", ",", "world", "!", "There", "is", "Datawhale", "."]

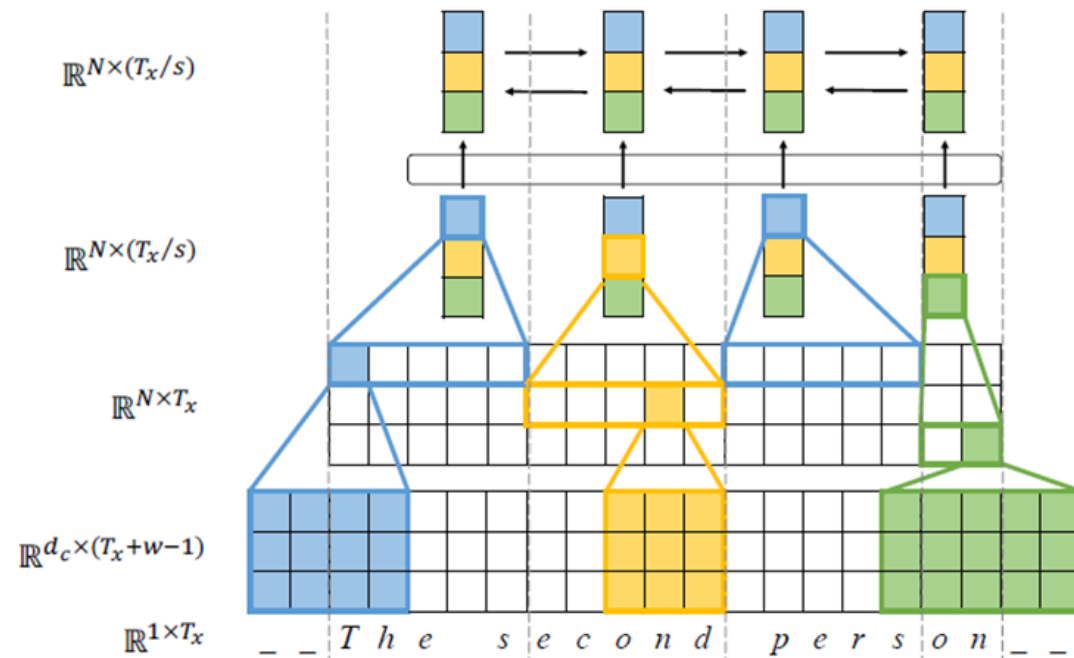
● 基于字符 (Character-based)

- 优点：无 OOV 问题、字符集灵活，适用于生成式任务或大量生僻词场景
- 缺点：缺乏语义信息、上下文理解难
- 示例：
 - 输入: "Hello"
 - 输出: ["H", "e", "l", "l", "o"]

Tokenizer的分类

● 基于子词 (Subword-based)

- 结合词级和字符级优势，常见算法包括 BPE、WordPiece 和 Unigram:
 - BPE (字节对编码)：合并高频字符对，平衡语义与灵活性
 - WordPiece：最大化子词概率，BERT 模型采用
 - Unigram：基于概率模型的最优分割
- 一般来说，子词Tokenizer兼顾词汇覆盖与语义保留，是LLM首选



Subword Tokenizer的常用算法

- **Byte Pair Encoding (BPE)**

- 原理：将频率高的字符对合并成新的子词，直到达到预定的词典大小或不再有高频的字符对。
- 示例：输入 “lower” ， 输出为 ["low", "er"]; 输入 “newest” ， 输出为 ["new", "est"]
- 特点：能有效处理词干和词缀，保持单词的基本语义结构。

- **WordPiece**

- 应用：BERT 模型采用该分词方式。
- 原理：与 BPE 不同，WordPiece 通过最大化子词在语言模型中的概率来选择子词，使分词结果更符合语言模型的预期。
- 示例：输入 “unhappiness” ， 输出为 ["un", "##happiness"], 其中 “##” 用于标记后续子词是单词的一部分。

- **Unigram**

- 原理：基于概率模型选择最优的子词分割方式。
- 示例：输入 “unhappiness” ， 输出为 ["un", "happiness"]; 输入 “newest” ， 输出为 ["new", "est"], 分割结果更符合语言使用习惯。

Transformers生态



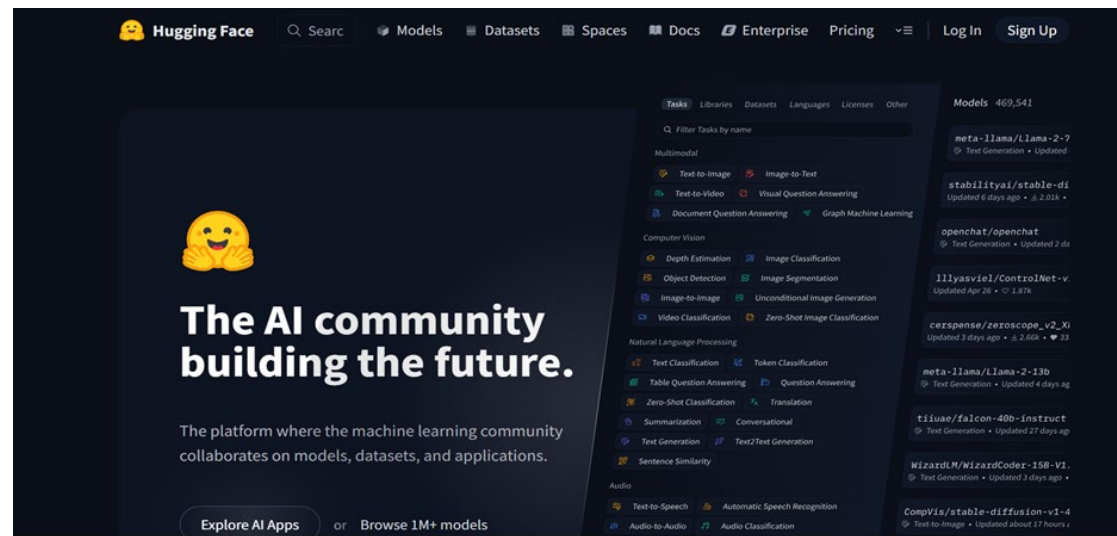
Transformers框架简介

● Transformers是当前LLM领域的主流训练和推理框架

- Transformers提供了数以千计的预训练模型，支持100多种语言的文本分类、信息抽取、问答、摘要、翻译、文本生成
- Transformers提供了便于快速下载和使用的API
- Transformers支持三个最热门的深度学习库Jax、PyTorch、Tensorflow并与之无缝整合



Hugging Face Transformers



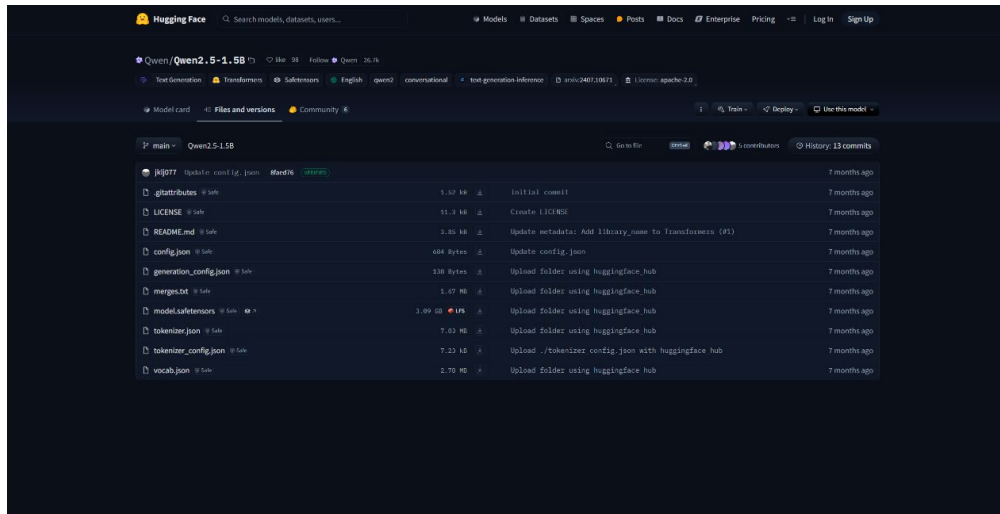
Hugging Face Transformers模型社区

初始化LLM

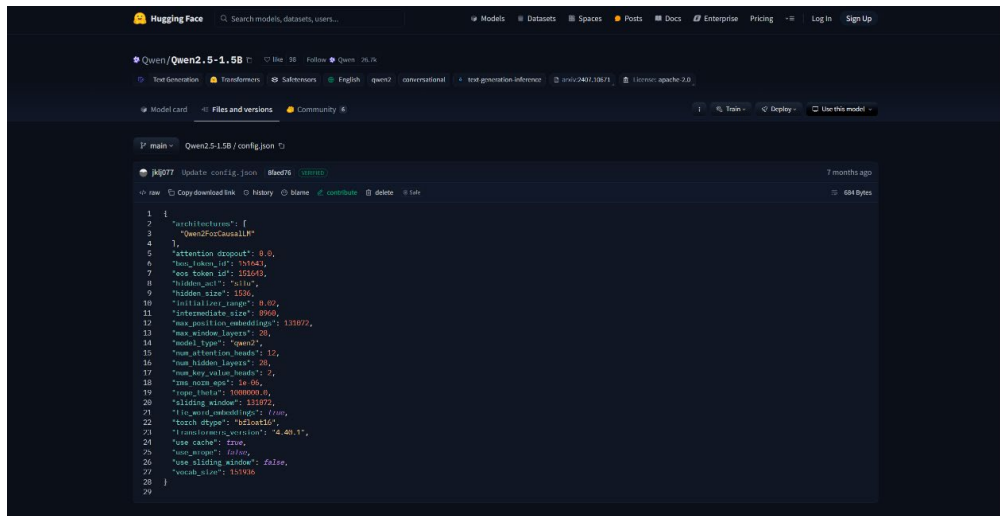
使用Transformers的AutoModel类来直接初始化已经实现好的模型。对于任意预训练模型，其参数中都包含有模型的配置信息。如果是想要从头训练一个LLM，可以使用一个已有的模型架构来直接初始化。这里，以Qwen-2.5-1.5B的模型架构为例：

查看模型配置参数:

- config.json文件是模型的配置信息
- 包括模型的架构、隐藏层大小、模型层数等



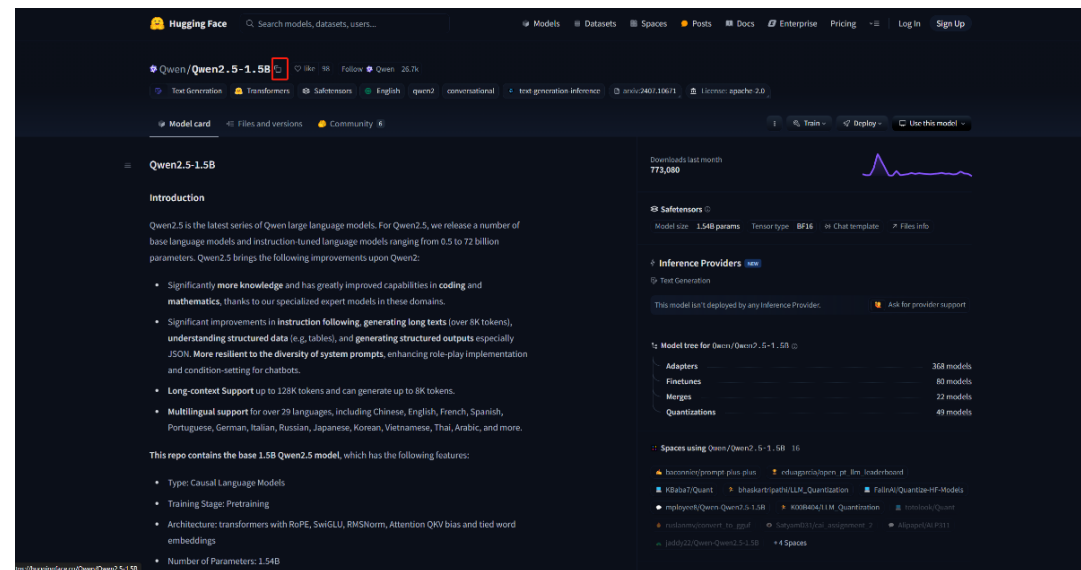
Qwen-2.5-1.5B



Qwen-2.5-1.5B config.json 文件

模型配置信息

```
import os
# 设置环境变量, 使用 HuggingFace 镜像网站
os.environ['HF_ENDPOINT'] = 'https://hf-mirror.com'
# 下载模型
os.system('huggingface-cli download --resume-download Qwen/Qwen2.5-1.5B --local-dir your_local_dir' )
```



模型下载标识

如右图，此处的Qwen/Qwen2.5-1.5B即为要下载模型的标识符，对于其他模型，可以直接复制HuggingFace上的模型名即可

加载默认配置

```
# 加载定义好的模型参数-此处以 Qwen-2.5-1.5B 为例
# 使用 transformers 的 Config 类进行加载
from transformers import AutoConfig
# 下载参数的本地路径
model_path = "qwen-1.5b"
config = AutoConfig.from_pretrained(model_name_or_path)
```

也可以对配置文件进行自定义，然后以同样的方式加载即可。可以使用 `AutoModel` 类基于加载好的配置对象生成对应的模型：

```
# 使用该配置生成一个定义好的模型
from transformers import AutoModelForCausalLM
model = AutoModelForCausalLM.from_config(config, trust_remote_code=True)
```

使用Trainer进行训练

使用 transformers 提供的 Trainer 类进行训练。

➤ TrainingArguments

```
from transformers import TrainingArguments
# 配置训练参数
training_args = TrainingArguments(
    output_dir="output", # 训练参数输出路径
    per_device_train_batch_size=4, # 训练的 batch_size
    gradient_accumulation_steps=4, # 梯度累计步数, 实际 bs = 设置的 bs * 累计步数
    logging_steps=10, # 打印 loss 的步数间隔
    num_train_epochs=1, # 训练的 epoch 数
    save_steps=100, # 保存模型参数的步数间隔
    learning_rate=1e-4, # 学习率
    gradient_checkpointing=True # 开启梯度检查点
)
```

模型微调



- **为何需要进行微调：**

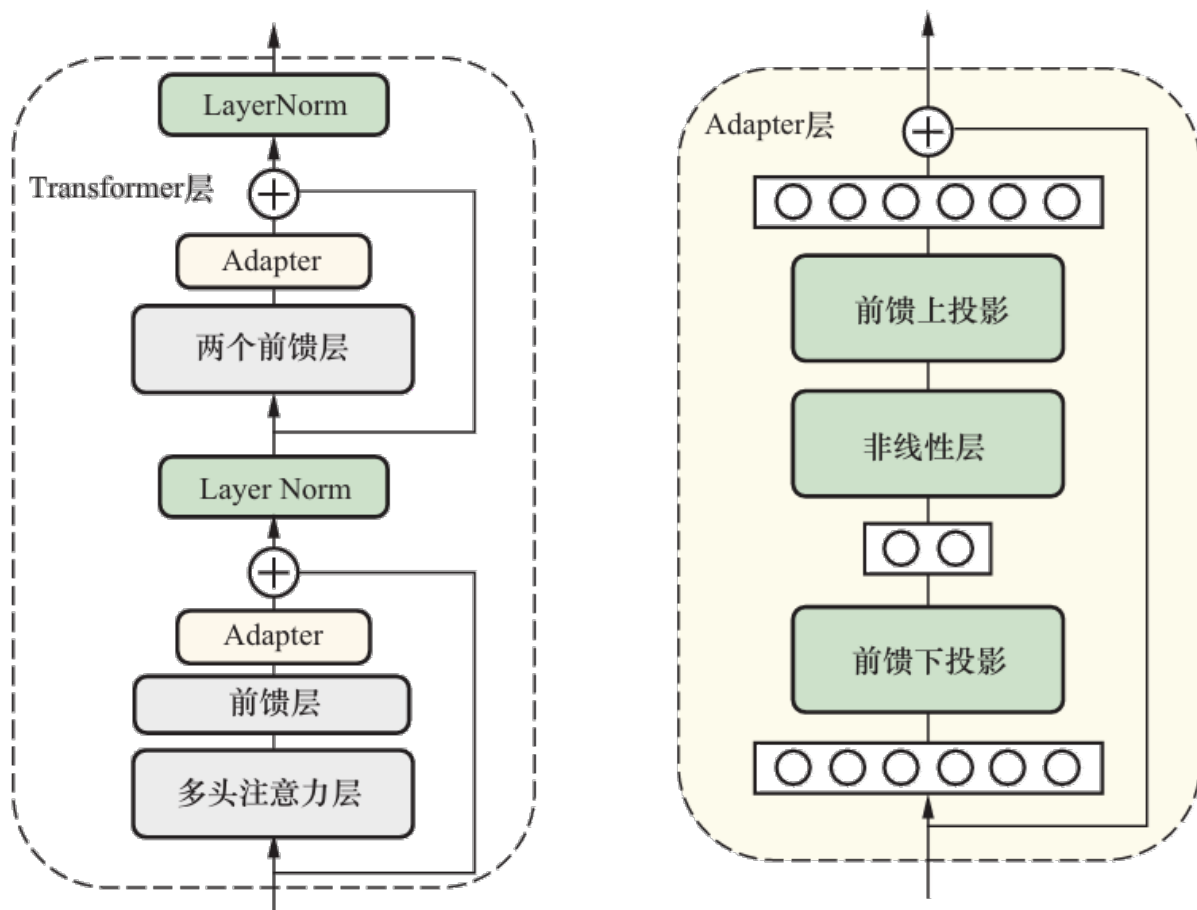
- 使LLM更好地完成特定目标任务。
- 由于LLM参数量大，训练数据多，对模型进行从头训练需要调整模型全部参数，资源压力非常大。
在资源有限的情况下，如何高效、快速对模型进行领域或任务的微调，以低成本地使用LLM完成目标任务，是非常重要的。

- **主要有两种解决方案：**

- Adapt Tuning：在模型中添加Adapter层，在微调时冻结原参数，仅更新Adapter层
- Prefix Tuning：固定预训练模型，为模型添加可训练，任务特定的前缀，这样就可以为不同任务保存不同的前缀，微调成本也小

Adapt Tuning

- Adapt Tuning在预训练模型每层中插入用于下游任务的参数，即Adapter模块，在微调时冻结模型主体，仅训练特定于任务的参数。



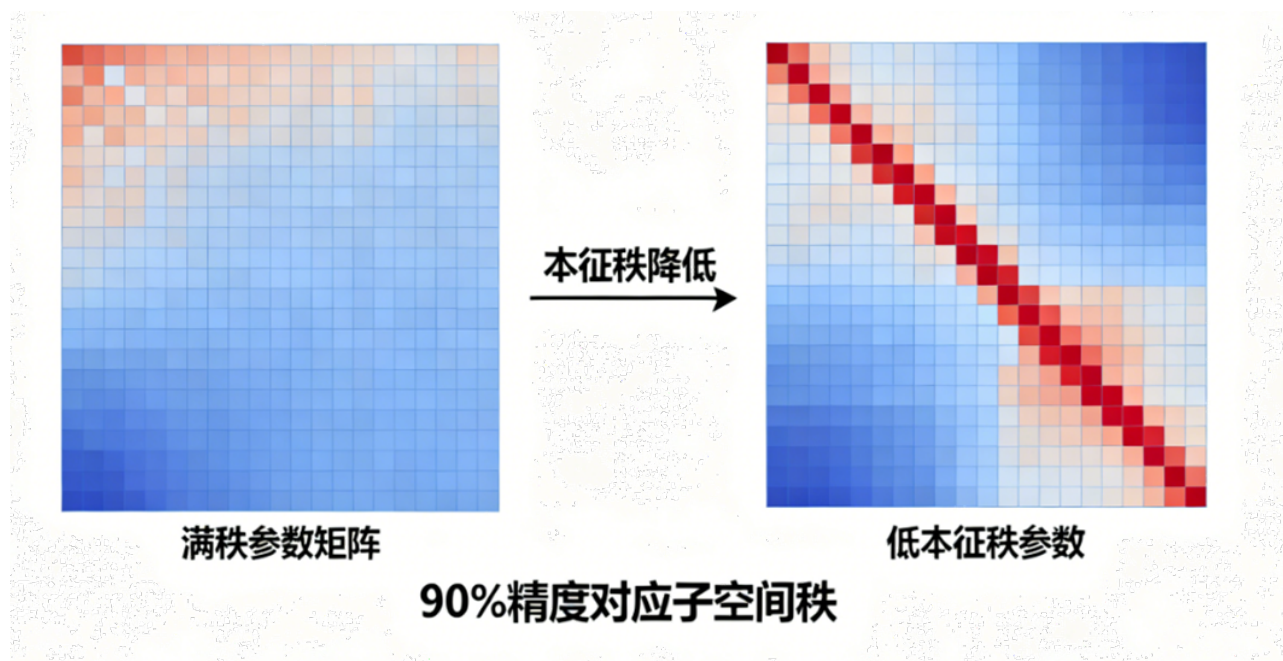
每个Adapter模块由两个前馈子层组成:

- 第一个前馈子层将Transformer块的输出作为输入，将原始输入维度 d 投影到 m ，通过控制 m 的大小来限制 Adapter 模块的参数量，通常情况下 $m \ll d$
- 第二个前馈子层还原输入维度，将 m 重新投影到 d ，作为 Adapter 模块的输出(如图右侧结构)

- LoRA事实上是一种改进的Adapt Tuning方法

- Adapt Tuning方法存在推理延迟问题，由于增加了额外参数和额外计算量，导致微调之后的模型计算速度相较原预训练模型更慢；
- 如果一个大模型是将数据映射到高维空间进行处理，那么在处理一个细分的小任务时，可能无需如此复杂的大模型，仅在某个子空间范围内就可以解决，从而也就不需要对全量参数进行优化了；
- 当对某个子空间参数进行优化时，如果能够达到全量参数优化性能的一定水平（如90%的精度），那么这个子空间参数矩阵的秩就可以称为对应当前待解决问题的**本征秩**（intrinsic rank）。

- LoRA事实上是一种改进的Adapt Tuning方法

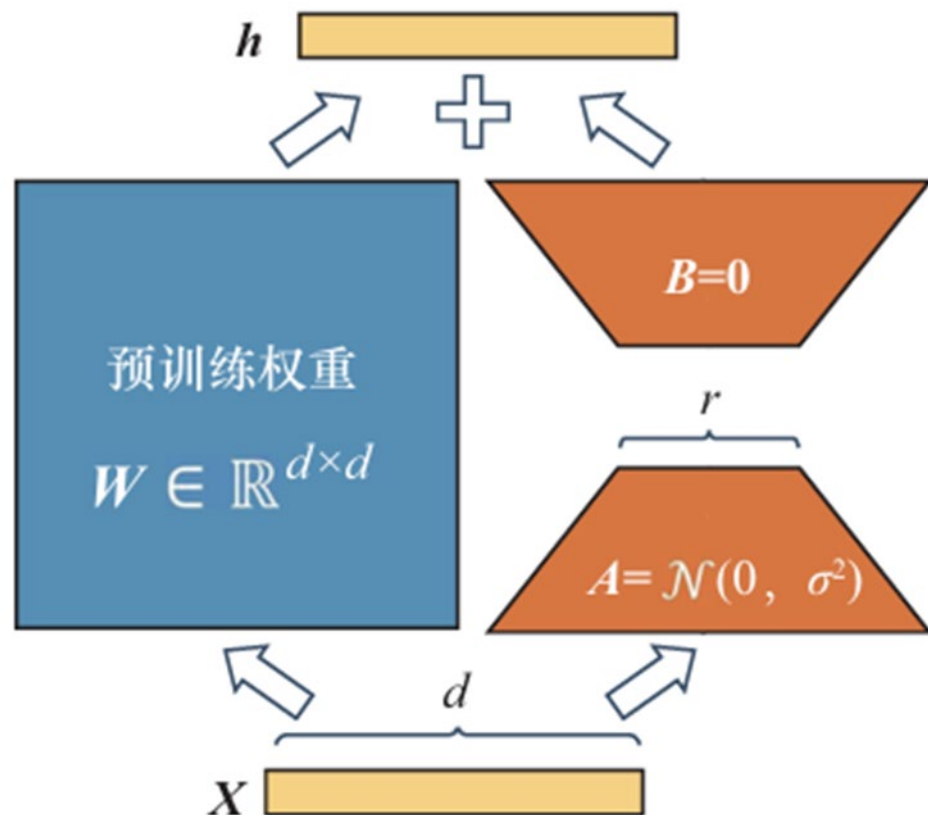


- 预训练模型本身就隐式地降低了**本征秩**，当针对特定任务进行微调后，模型中权重矩阵其实具有更低的本征秩。同时，越简单的下游任务，对应的本征秩越低。
- 假设预训练参数为 θ_0^D ，在特定下游任务上密集层权重参数矩阵对应的**本征秩**为 θ^d ，对应特定下游任务微调参数为 θ^D ，那么有：

$$\theta^D = \theta_0^D + \theta^d M$$

- 这个 M 即为 LoRA 优化的**秩分解矩阵**。

• 低秩参数化更新矩阵



- LoRA假设权重更新的过程中也有一个较低的本征秩，对于预训练的权重参数矩阵 $W_0 \in \mathbb{R}^{d \times k}$ (d 为上一层输出维度, k 为下一层输入维度), 使用低秩分解来表示其更新:

$$W_0 + \Delta W = W_0 + BA \text{ where } B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times k}$$

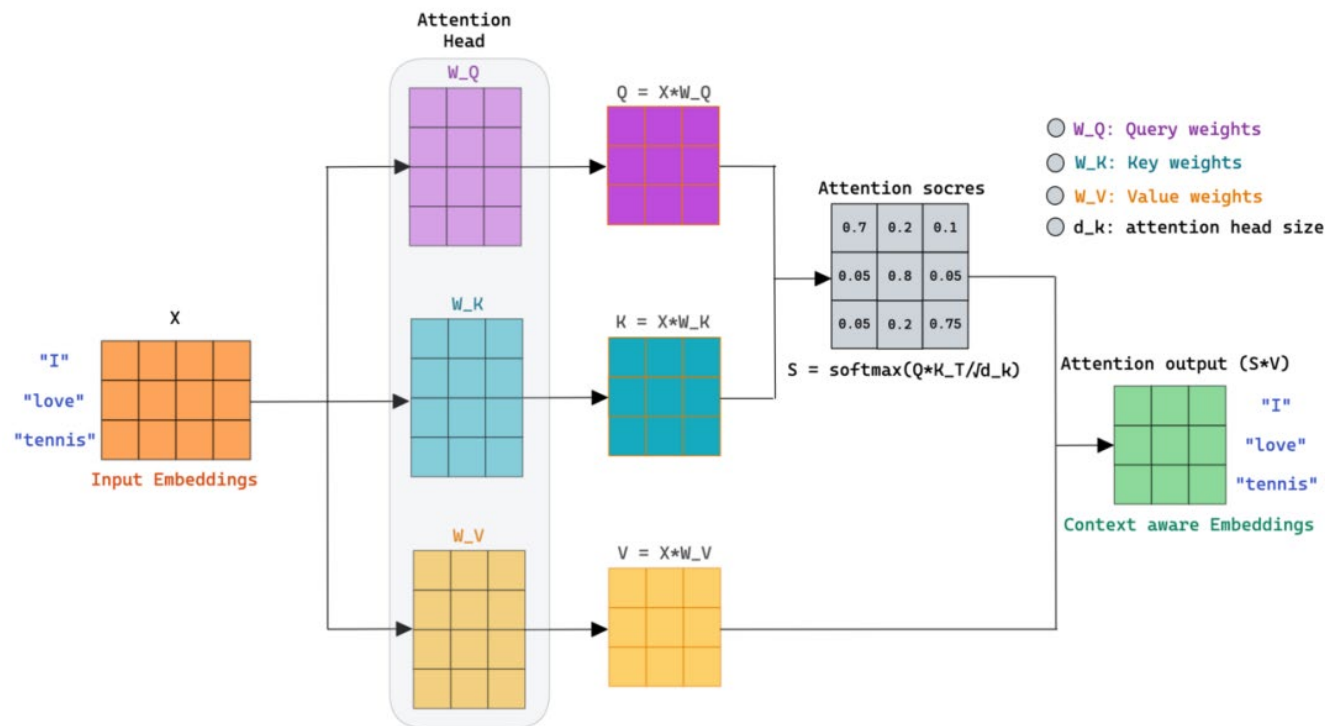
- 在训练过程中, W_0 冻结不更新, A 、 B 包含可训练参数。
- 因此, LoRA的前向传递函数为:

$$h = W_0 x + \Delta W x = W_0 x + BAx$$

- 在开始训练时, 对 A 使用随机高斯初始化, 对 B 使用零初始化, 然后使用Adam进行优化。

• 应用于Transformer

- 在Transformer结构中，LoRA技术主要应用在注意力模块的四个权重矩阵： W_q 、 W_k 、 W_v 、 W_o ，而冻结MLP的权重矩阵



- 权重矩阵 (W_q 、 W_k 、 W_v) 是可训练的参数矩阵，通过线性变换将输入数据映射到 Q、K、V 向量空间。
- 通过消融实验发现同时调整 W_q 和 W_v 会产生最佳结果。
- 在上述条件下，可训练参数个数为：

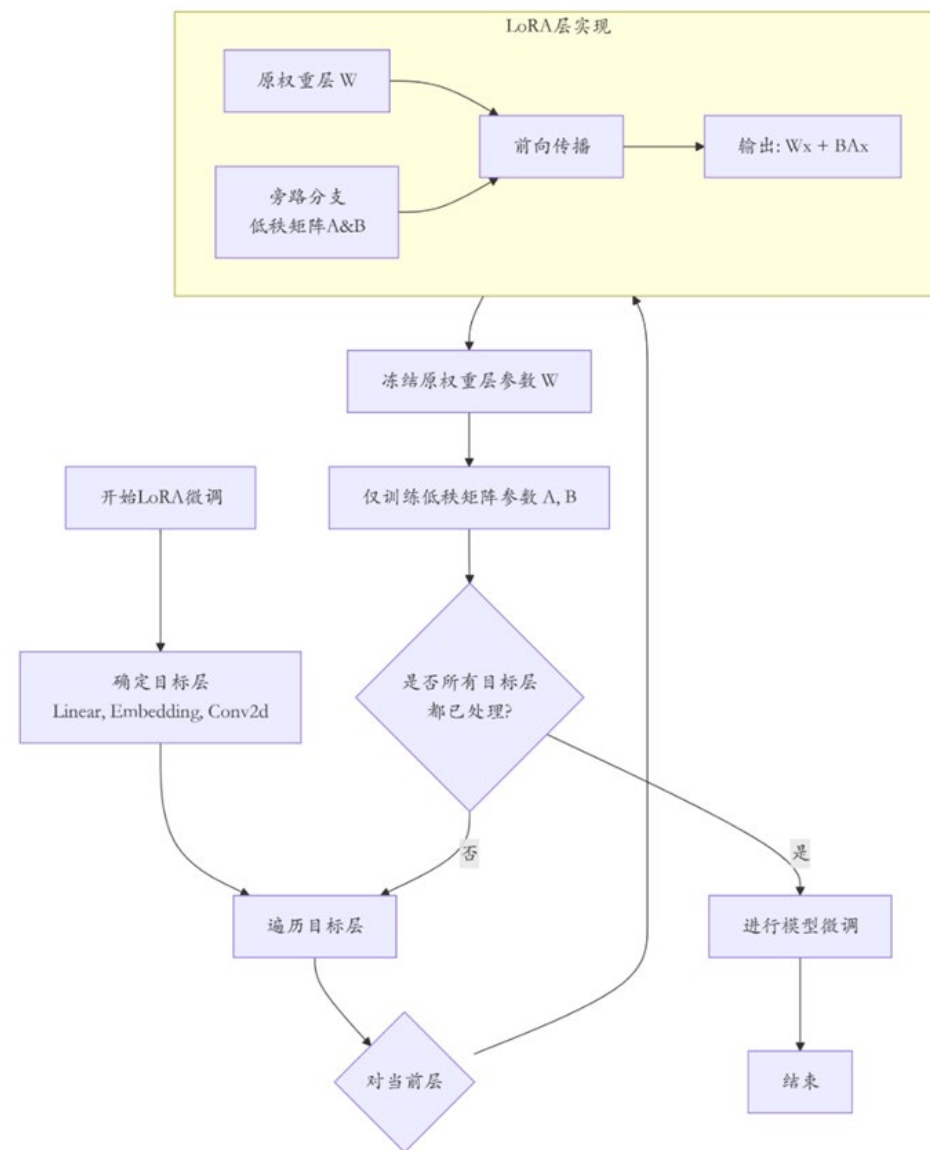
$$\Theta = 2 \times L_{LoRA} \times d_{model} \times r$$

LoRA微调

- 目前一般通过peft库来实现模型的LoRA微调

- 实现流程

- 确定要使用LoRA的层peft库目前支持调用LoRA的层包括：nn.Linear、nn.Embedding、nn.Conv2d 三种。
- 对每一个要使用LoRA的层，替换为LoRA层所谓LoRA层，实则是在该层原结果基础上增加了一个旁路，通过低秩分解（即矩阵 A 和矩阵 B ）来模拟参数更新。
- 冻结原参数进行微调，更新LoRA层参数。



检索增强生成 (RAG)



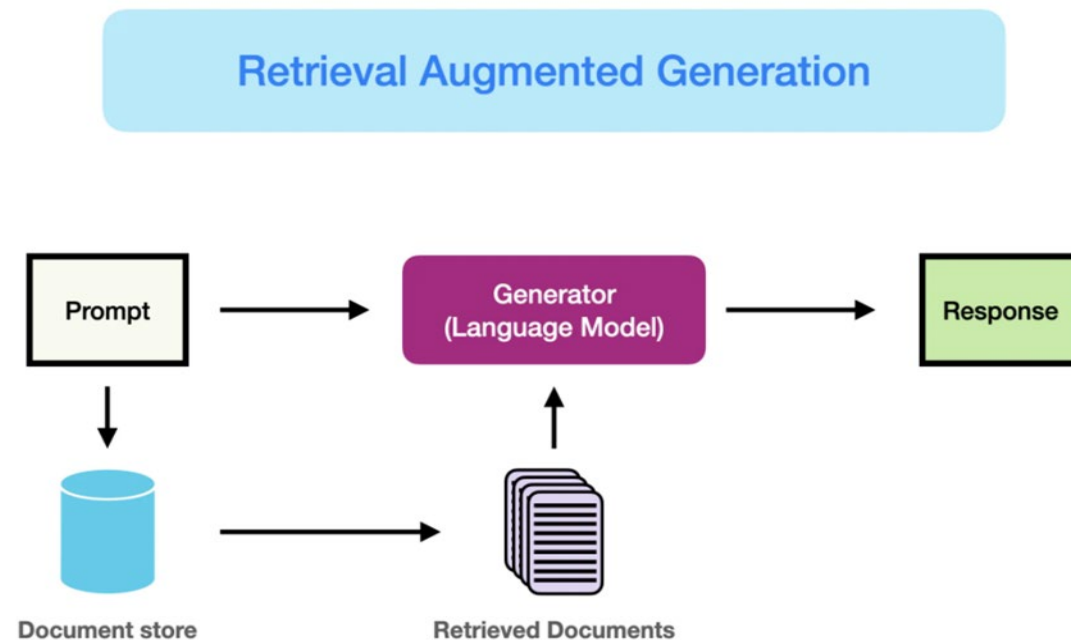
RAG的基本原理

- **幻觉：大语言模型在生成内容时产生的不准确或误导性的信息**

- 幻觉的核心原因：LLM回答问题所需的信息不足
- 解决幻觉的方法是引入实时外部知识源

- **检索增强生成（RAG）：**

- 从外部的数据库中检索出相关信息
- 将这些信息融入到生成过程，从而指导和优化语言模型的输出



RAG的基本原理

- **幻觉的核心原因：**
LLM回答问题所需的信息不足
- **更确切的说法是：**
数据偏差与模型基于概率推测的生成本质

- **数据偏差：**
模型在预训练时用的数据**错误**或者**过时**
- **概率推测：**
模型的下一个token的输出是基于前面的token,它会计算哪个token接在后面的**概率最高**。只要概率算得通，它就能一本正经地胡说八道。

User Input



Can you recommend a delicious recipe for dinner?

LLM Response



Yes, here is a delicious recipe for **lunch**. So how about fried chicken with mashed potatoes? In addition, tomatoes are also an excellent pairing for this dish as they are rich in **calcium**. Enjoy this **steak**!

Hallucination Explanation

Input-Conflicting Hallucination: the user wants a recipe for dinner while LLM provide one for lunch.

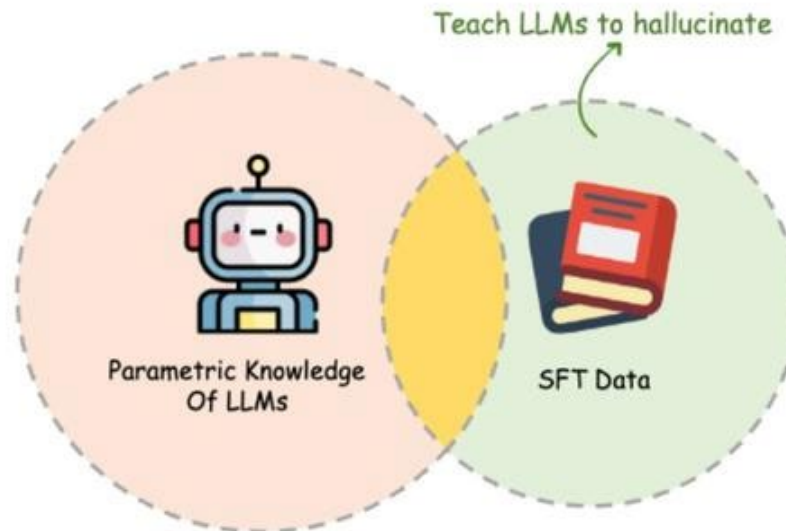
Context-Conflicting Hallucination: steak has not been mentioned in the preceding context.

Fact-Conflicting Hallucination: tomatoes are not rich in calcium in fact.

RAG的基本原理

- **解决幻觉的方法：**
引入**实时外部知识源**
- **更确切的说法是：**
RAG技术，并结合**人类反馈强化学习（RLHF）**来减少错误输出。

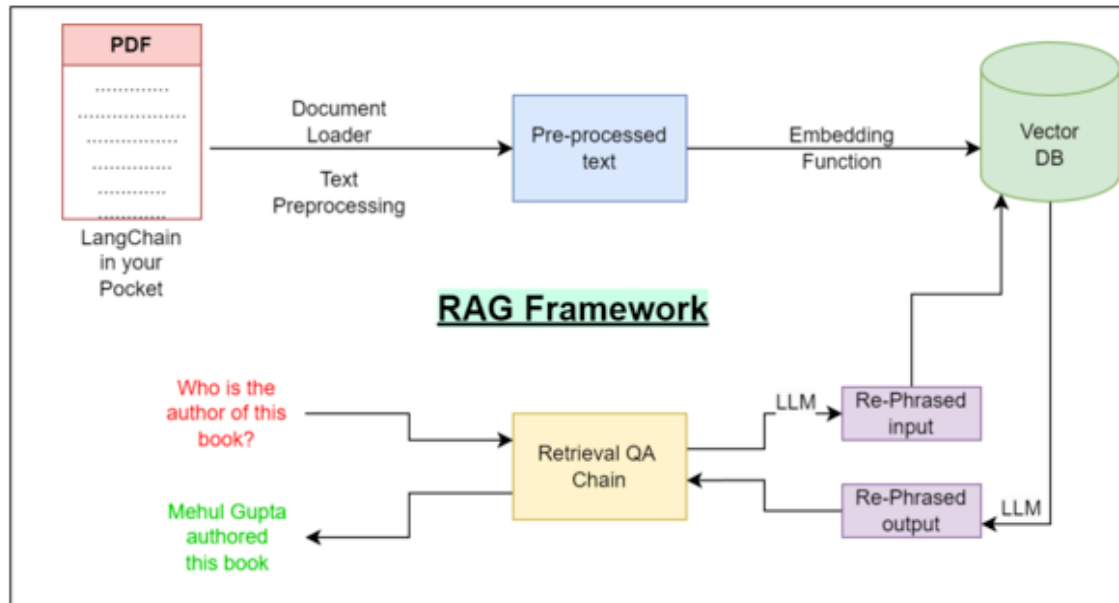
- **RAG技术：**
引入**外部知识源**进行实时校正，通过验证性数据来支持和约束生成过程
- **RLHF**
RLHF 通过引入人类的判断标准，给模型增加了“**真实性**”的约束



RAG的基本原理

➤ RAG 可分为 “检索” “增强” “生成” 三个阶段：

- **检索：**检索是RAG流程的第一步，从预先建立的知识库中检索与问题相关的信息
- **增强：**将检索到的信息用作生成模型的上下文输入，以增强模型对特定问题的理解和回答能力
- **生成：**生成器会利用检索到的信息作为上下文输入，并结合大语言模型来生成符合用户需求的回答



RAG (检索增强生成)
是一种 AI 框架，它将传统信息检索系统（例如数据库）的优势与生成式大语言模型 (LLM) 的功能结合在一起。

搭建RAG框架

● RAG基本流程

RAG的基本结构

- **向量化模块**：用来将文档片段向量化
- **文档加载和切分模块**：用来加载文档并切分成文档片段
- **数据库**：存放文档片段及其对应的向量表示
- **检索模块**：根据 Query（问题）检索相关的文档片段
- **大模型模块**：根据检索到的文档回答用户的问题



搭建RAG框架



为什么选择 MILVUS  文档 教程  工具  博客 社区 

简体中文  Star 38.4K 联系我们

搜索

首页 v2.6.x 

关于 Milvus

开始

快速入门

安装 Milvus

安装 SDK

连接 Milvus 服务器

概念

用户指南

数据导入

管理指南

工具

集成

教程

常见问题

API Reference

首页 > 文档 > 开始 > 快速入门

快速入门 Milvus Lite

Open in Colab View on Github

向量是神经网络模型的输出数据格式，可以有效地对信息进行编码，在知识库、语义搜索、检索增强生成（RAG）等人工智能应用中发挥着举足轻重的作用。

Milvus 是一个开源的向量数据库，适合各种规模的人工智能应用，从在 Jupyter notebook 中运行一个演示聊天机器人，到构建服务数十亿用户的网络规模搜索。在本指南中，我们将指导您如何在几分钟内本地设置 Milvus，并使用 Python 客户端库生成、存储和搜索向量。

安装 Milvus

在本指南中，我们使用 Milvus Lite，它是 `pymilvus` 中包含的一个 python 库，可以嵌入到客户端应用程序中。Milvus 还支持在 [Docker](#) 和 [Kubernetes](#) 上部署，适用于生产用例。

开始之前，请确保本地环境中有 Python 3.8+ 可用。安装 `pymilvus`，其中包含 python 客户端库和 Milvus Lite：

```
$ pip install -U pymilvus
```

NOTE

翻译自 DeepL

目录

快速入门 Milvus Lite

安装 Milvus

设置向量数据库

创建 Collections

准备数据

用向量表示文本

[另一种方法] 使用随机向量的假表示法

插入数据

语义搜索

带元数据过滤的向量搜索

删除实体

加载现有数据

删除 Collections

了解更多



想要更快、更简单、更好用的 Milvus SaaS 服务？

Zilliz Cloud 是基于 Milvus 的全托管向量数据库，拥有更高性能，更易扩展，以及卓越性价比。

免费试用 Zilliz Cloud

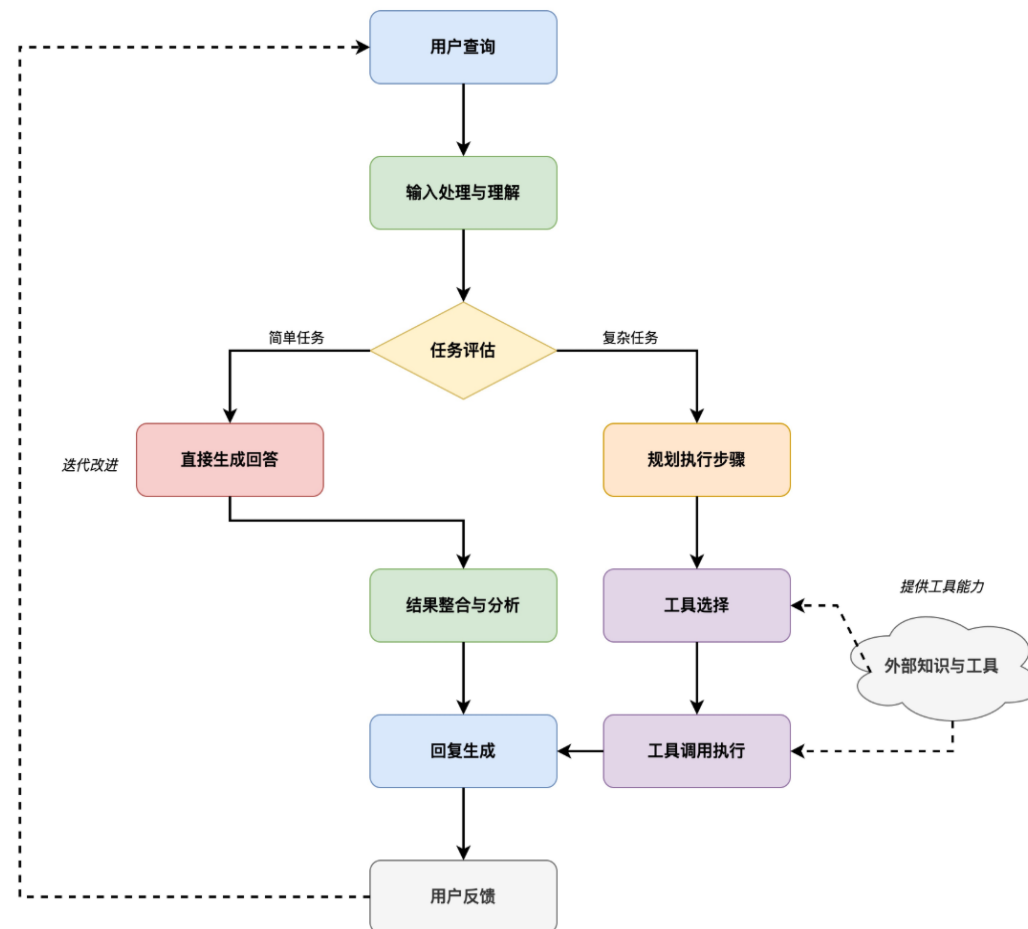
Agent



什么是LLM Agent

大模型Agent是一个以LLM为核心“大脑”，并赋予其自主规划、记忆和使用工具能力的系统

- **理解目标：**接收一个相对复杂或高层次的目标
- **自主规划：**将大目标分解成一系列可执行的小步骤
- **记忆：**拥有短期记忆和长期记忆
- **工具使用：**调用外部API、插件或代码执行环境来获取信息
- **反思与迭代：**能够评估自己的行为 and 结果，从中学习并调整后续计划。



LLM Agent的类型

根据Agent设计理念和能力侧重，我们可以大致将其分为几类：

- **任务导向型Agent**：专注于完成特定领域的、定义明确的任务
- **规划与推理型Agent**：强调自主分解复杂任务、制定多步计划
- **多Agent系统**：由多个具有不同角色或能力的Agent协同工作，共同完成一个更宏大的目标
- **探索与学习型Agent**：不仅执行任务，还能在与环境的交互中主动学习新知识、新技能或优化自身策略

LLM Agent!



动手构造一个Tiny-Agent

➤ Step 1 : 初始化客户端和模型

我们需要一个能够调用大模型的客户端，使用 openai 库，并配置其指向一个兼容 OpenAI API 的服务终端，同时，指定要使用的模型，如 Qwen/Qwen2.5-32B-Instruct

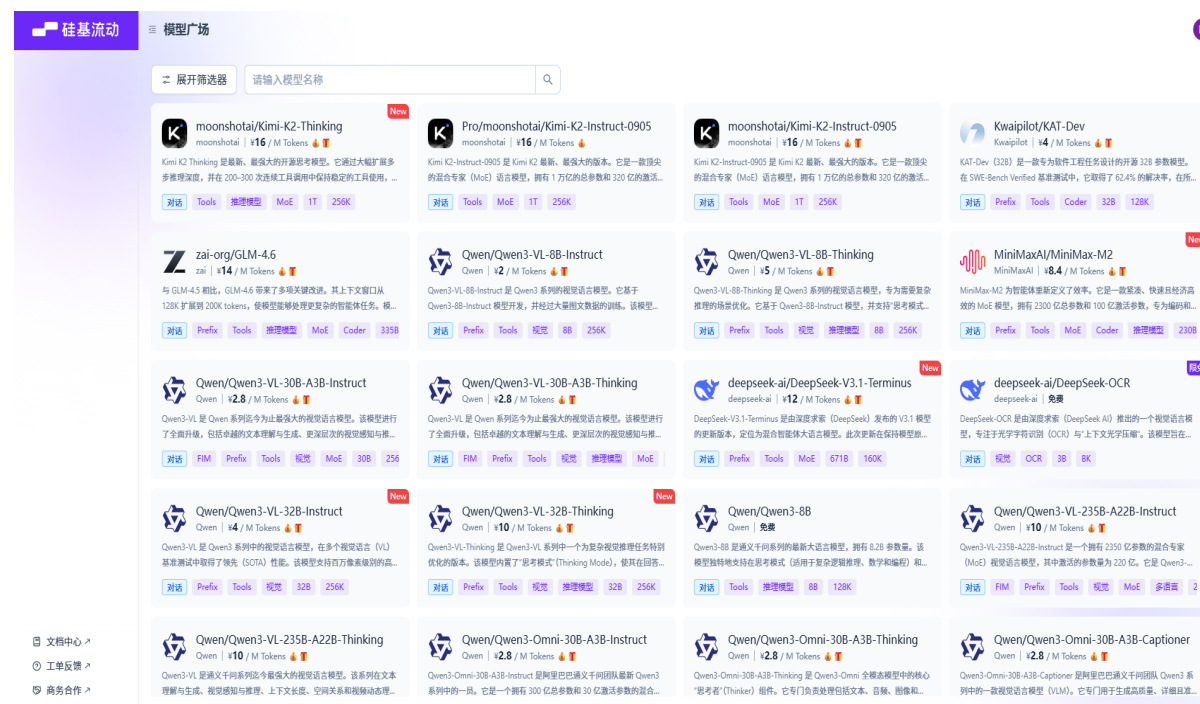
```
from openai import OpenAI
```

```
# 初始化 OpenAI 客户端
```

```
client = OpenAI(  
    api_key="YOUR_API_KEY", # 替换为你的Key  
    base_url="https://api.siliconflow.cn/v1",  
    # 使用模型访问的API地址  
)
```

```
# 指定模型名称
```

```
model_name = "Qwen/Qwen2.5-32B-Instruct"
```



动手构造一个Tiny-Agent

➤ Step 2: 定义工具函数

文件中定义 Agent 可以使用的工具函数。每个函数都需要有清晰的文档字符串描述其功能和参数，因为这将用于自动生成工具的 JSON Schema

```
# src/tools.py
from datetime import datetime

# 获取当前日期和时间
def get_current_datetime() -> str:
    """
    获取当前日期和时间。
    :return: 当前日期和时间的字符串表示。
    """
    current_datetime = datetime.now()
    formatted_datetime =
current_datetime.strftime("%Y-%m-%d
%H:%M:%S")
    return formatted_datetime
```

```
def function_to_json(func) -> dict:
    # ... (函数实现细节)
    # 返回符合 OpenAI tool schema 的字典
    return {
        "type": "function",
        "function": {
            "name": func.__name__,
            "description": inspect.getdoc(func),
            "parameters": {
                "type": "object",
                "properties": parameters,
                "required": required,
            },
        },
    }
```

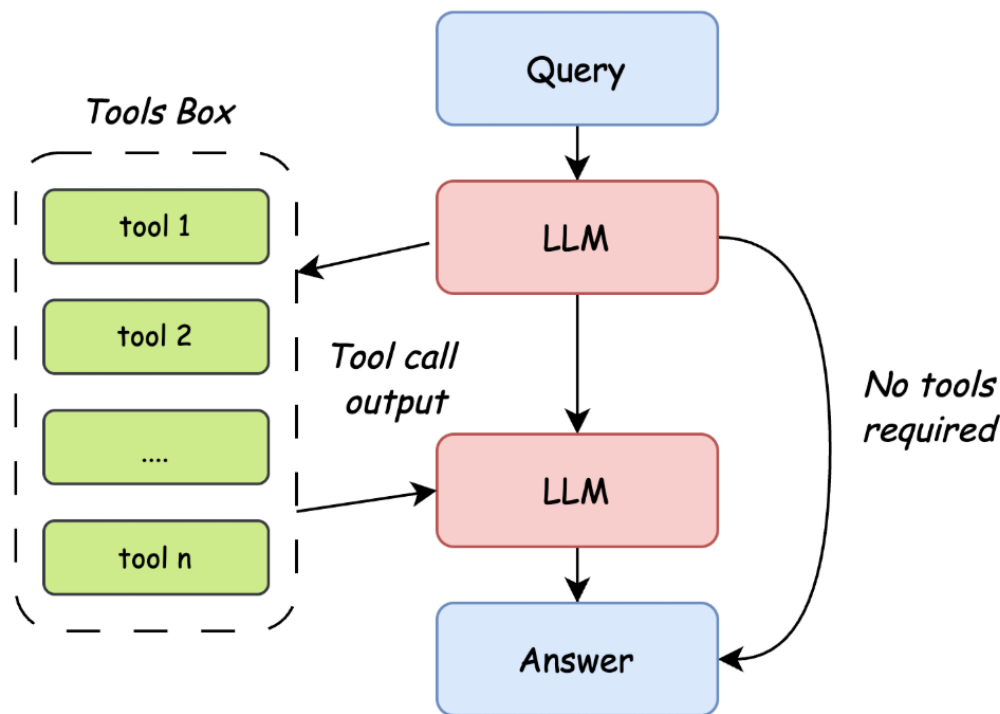
动手构造一个Tiny-Agent

➤ Step 3: 构造 Agent 类

Agent 类负责管理对话历史、调用 OpenAI API、处理工具调用请求以及执行工具函数

Agent 的工作流程如下:

1. 接收用户输入。
2. 调用大模型（如 Qwen），并告知其可用的工具及其 Schema。
3. 如果模型决定调用工具，Agent 会解析请求，执行相应的 Python 函数。
4. Agent 将工具的执行结果返回给模型。
5. 模型根据工具结果生成最终回复。
6. Agent 将最终回复返回给用户。



[快速开始](#) [安装](#) [消息渠道](#) [代理](#) [工具](#) [模型](#) [平台](#) [网关与运维](#) [参考](#) [帮助](#)

概览

[展示专区](#)

核心概念

[功能](#)

第一步

[入门指南](#)

[新手引导 \(CLI\)](#)

[新手引导 \(macOS 应用\)](#)

指南

[个人助理设置](#)

[快速开始](#) / [OpenClaw](#)

[快速开始](#)

OpenClaw

[Copy page](#)

[Edit source](#)



“脱壳！脱壳！” – 大概是一只太空龙虾

适用于任意操作系统的 AI 智能体 Gateway 网关，可跨 Discord、Google Chat、iMessage、Matrix、Microsoft Teams、Signal、Slack、Telegram、WhatsApp、Zalo 等使用。

发送一条消息，就能从口袋里收到智能体回复。在内置渠道、捆绑渠道插件、WebChat 和移动节点之间运行一个 Gateway 网关。



开始使用

安装 OpenClaw，并在几分钟内启动 Gateway 网关。



运行新手引导

使用 `openclaw onboard` 和配对流程完成引导式设置。



打开控制 UI

启动用于聊天、配置和会话的浏览器仪表盘。

≡ On this page

[什么是 OpenClaw?](#)

[工作原理](#)

[核心能力](#)

[快速开始](#)

[仪表盘](#)

[配置 \(可选\)](#)

[从这里开始](#)

[了解更多](#)

► 开源 · MIT 许可证 · Nous Research 出品

Hermes Agent

随你成长的 AI 智能体。

部署在你的服务器上，连接你的消息账号，它就成为你的持久个人智能体——学习你的项目、自动构建技能、随时随地触达你。不是聊天机器人，不是代码补全工具，而是一个住在你机器上、每天都在变聪明的智能体。

免费开始使用 →

了解更多

40+

内置技能

5

聊天平台

MIT

开源许可

100%

自托管

一个智能体，满足所有需求

Hermes Agent 将持久记忆、自动技能创建和多平台接入整合在一个开源包中。

Q&A